

Raport – tema 2 – IA

3.1 – Vizualizarea augmentarii seturilor de date

Pentru a antrena o retea neuronală cât mai bine, avem nevoie de cât mai multe date. Astfel, aplicăm mai multe augmentări asupra pozelor pentru a obține noi date pentru fiecare epocă. Iar modelul reușește să învețe ceva nou mereu, nu va intra într-un ciclu unde învață aceleași date și ulterior să ajungă la overfitting. Ce trebuie însă să respectăm este să nu distorsionăm fotografiile foarte grav sau în funcție de caz să aplicăm augmentări care să dezavantajeze ceea ce trebuie modelul să învețe (de ex: nu putem face flip sus-jos pentru fete, deoarece s-ar schimba expresia facială, modelul învățând date eronate).

Augmentări pentru alunite:

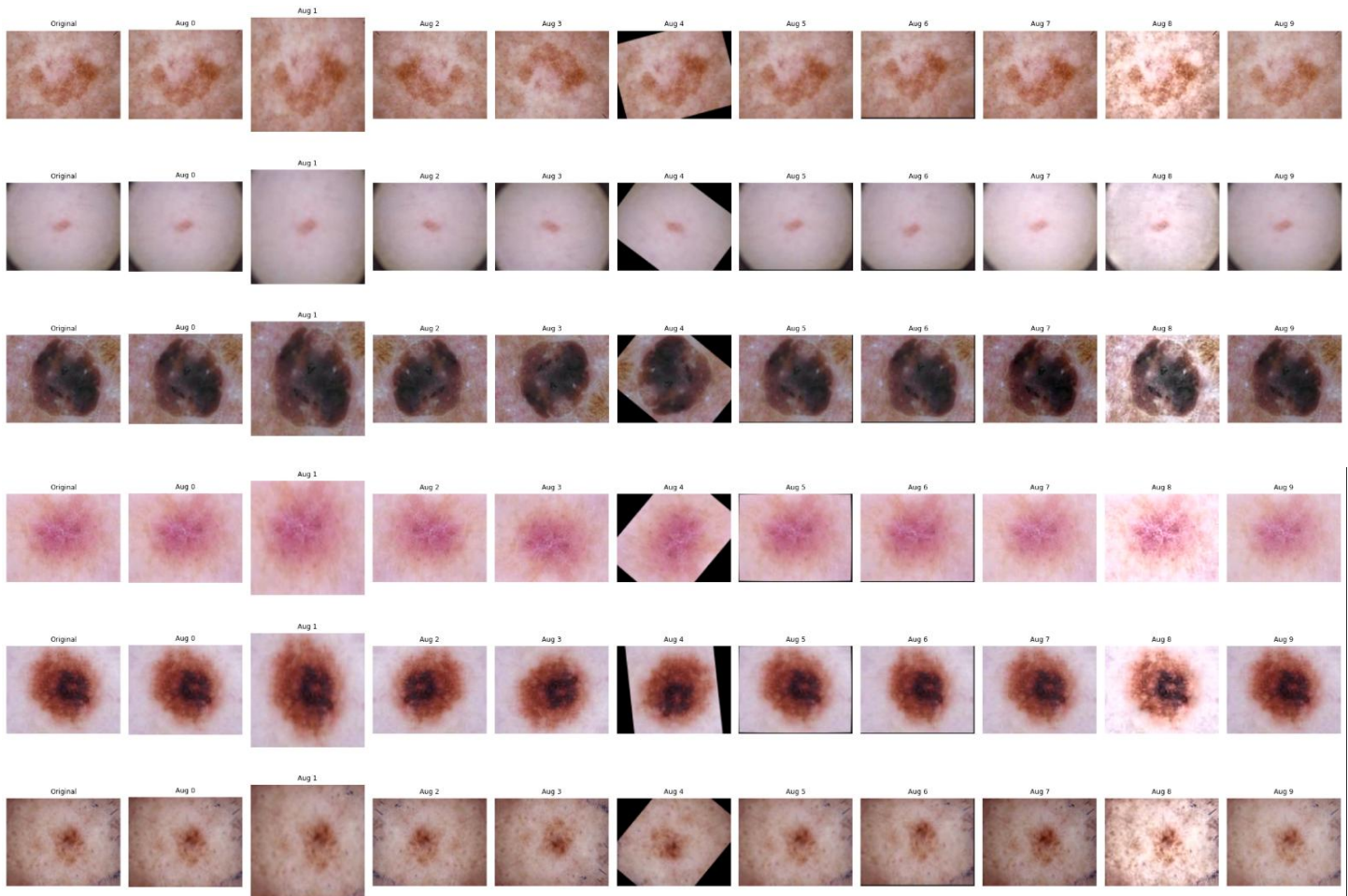
- *Resize si RandomResizedCrop*: redimensionez imaginea, deoarece cadrul poate fi mai aproape sau mai departe
- *HorizontalFlip si VerticalFlip*: întorc imaginea pe mai multe părți, deoarece în acest caz o alunite întoarsă spre stanga, dreapta, sus sau jos nu influențează rezultatul, tot aceeași alunite rămâne. Cadrul imaginii poate din diverse unghiuri, iar aceste augmentări testează aceste cazuri
- *Rotate*: rotește imaginea, la fel ca mai sus, cadrul poate fi din unghiuri diferite
- *OpticalDistortion si GridDistortion*: distorsionează imaginea într-un mod subtil. Eu am făcut analogia cu momentul în care medicul oftamolog reglează aparatul unde pacientul vede o imagine cu un balon în departare. În timp ce reglează, pacientul vede balanoul I ceata, fie mai aproape, fie mai departe etc. Astfel aceste augmentări ajută modelul să identifice și poze care nu sunt perfecte
- *RandomBrightnessContrast, Clahe, RandomGamma*: ajustează luminozitatea, contrastul, aplicând transformări foarte mici, aproape invizibile, deoarece la alunite conține foarte mult culoarea pentru a fi clasificate, dar la fel identifică diverse situații și mediul în care a fost făcută poza
- *Normalize*: folosesc media și deviația standard din enunț, practic transformă poza la o scară standard, ajutând modelul să aibă un parcurs de învățare mai stabil

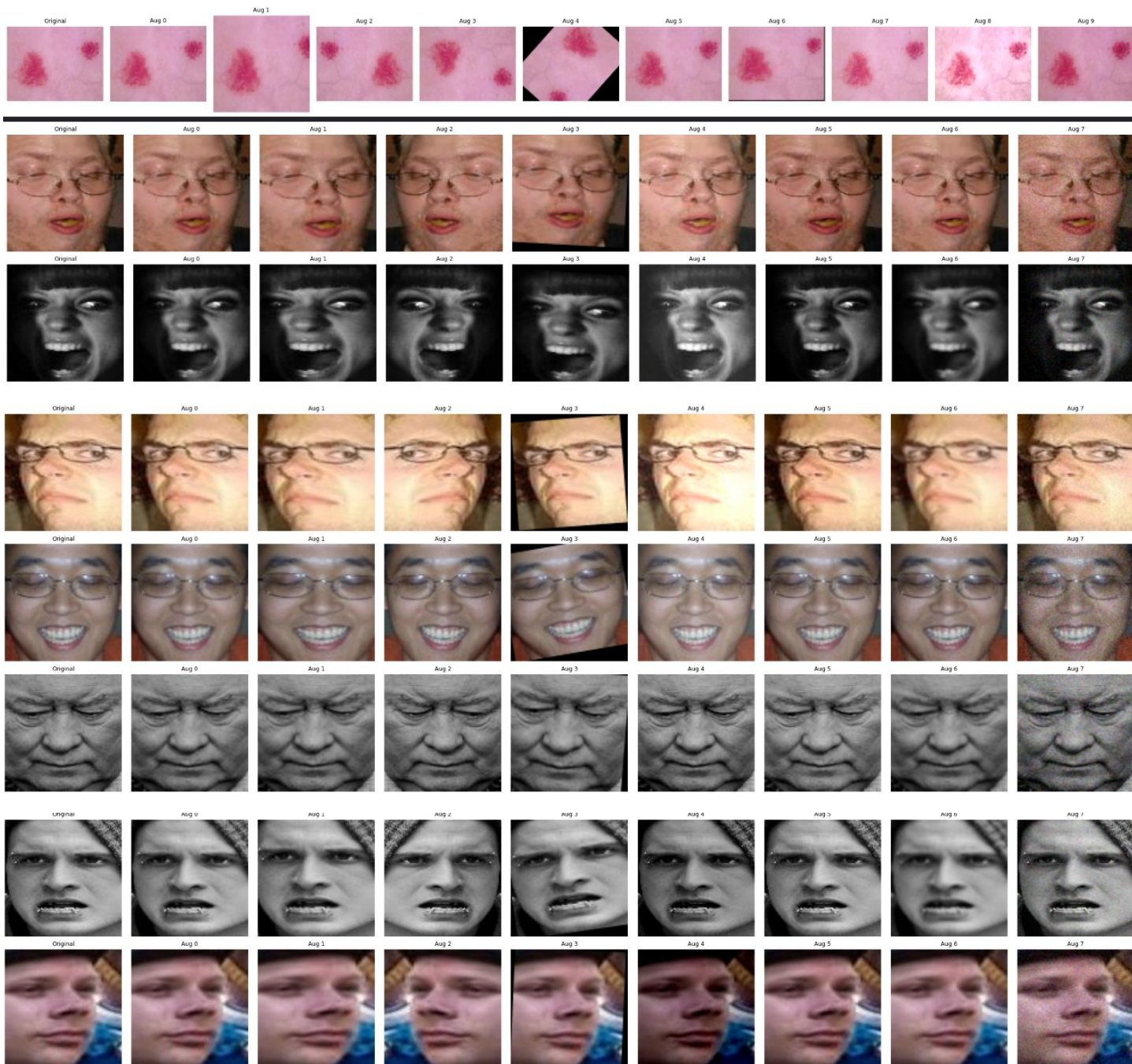
Augmentări pentru fete:

- *Resize si RandomResizedCrop*: la fel ca la alunite
- *HorizontalFlip*: dacă întorc stanga-dreapta, expresia facială este aproximativ aceeași, în teorie, flip sus-jos a fost exclus, deoarece modelul ar fi învățat gresit, în mod normal o expresie facială este cu fața așa cum o vedem în viața reală, nu cu susul în jos
- *Affine*: translații, scalar, rotații subtile, adică nu rotesc, de exemplu, ca la alunite la un unghi de 180 de grade. Mă ajută aceste transformări pentru a marca faptul că o poză nu are mereu fața perfect centrată, în aceeași poziție. Poate fi mai spre dreapta sau stanga, mai într-o parte etc

- *RandomBrightnessContrast, RandomGamma*: la fel ca la alunite
- *GaussianBlur si GaussNoise*: prima ofera blur, iar a doua zgomot, m-am gandit la tipuri de camera, unele mai bune, unele mai proaste, de exemplu putem face o comparatie intr-o imagine 4k si una 720p si tot asa, iar la zgomot, m-am gandit la o poza facuta la lumina buna in comparatie cu una facuta noaptea
- *Normalize*: tot ca la alunite

Mai jos, am aplicat fiecare augmentare pe cate un exemplu de imagine din fiecare categorie. Putem observa mici schimbari, nu sunt modificari bruste, radicale, ci transformari usoare care sa arate mai multe informatii, mai multe situatii.





Am ales augmentările în funcție de situație. Dacă la alunele m-am putut juca cu rotirea imaginii, la fete nu am putut să fac asta într-un mod impactant. Ce vreau să spun este că am încercat să transpun imaginea în funcție de categorii (alune vs fete) în situații cât mai diversificate pentru a ajuta modelul să învețe cât mai mult.

3.2 – Arhitectura MLP

Am făcut analogia între MLP și un puzzle, practic imaginea este împartită în pixeli, piese de puzzle, iar apoi fiecare pixel se va duce într-o categorie, folosind weightul, care este un scor, iar acel scor îl duce într-o anumită categorie. La fel ca la puzzle, unde ordonăm piesele, de exemplu, după forme, la fel și aici sunt ordonați pixelii pe categorii, iar la fiecare epocă, rețeaua învață din greșeli și taxează să zică așa scorul ce îl propulsa într-o categorie. Faptul că rețeaua nu

stie ca pixelii formeaza o imagine completa, face clasificarea mult mai dificila pentru aceasta arhitectura.

In ceea ce priveste arhitectura MLP am ales urmatoarele straturi:

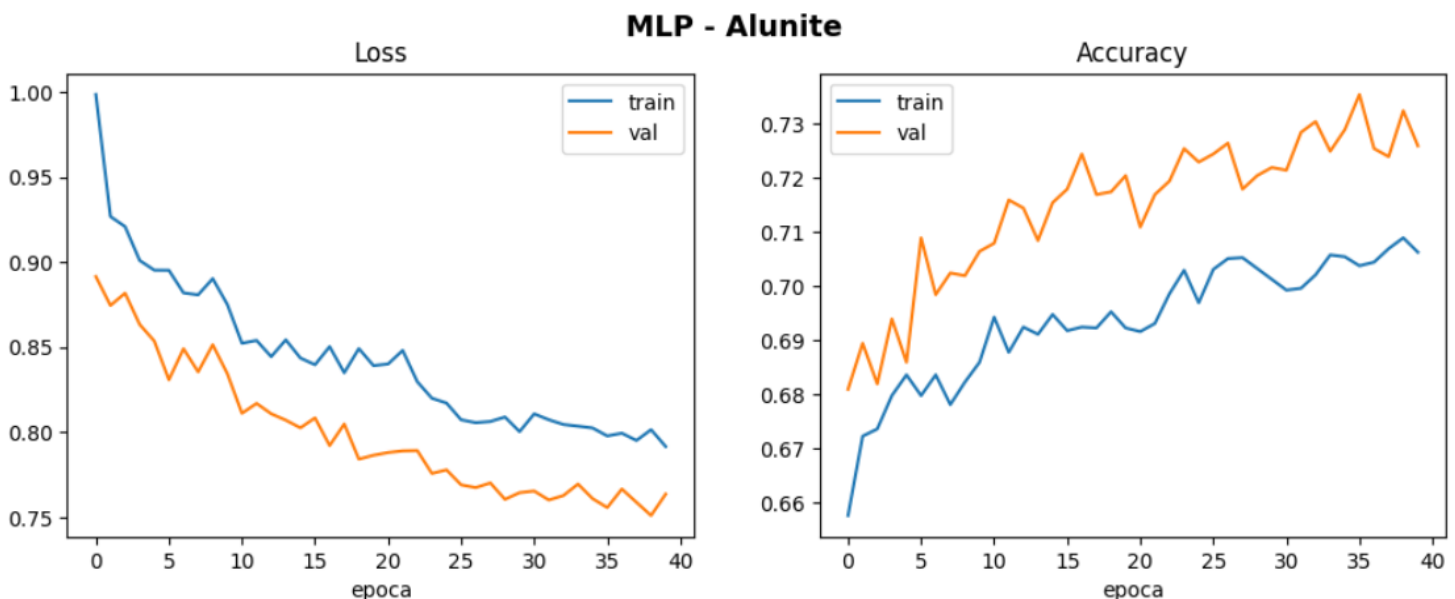
- `nn.Flatten()`: transforma imaginea 3D in vector 1D
- `nn.Linear(input_size, 512)`: combina pixelii in 512 combinatii
- `nn.BatchNorm1d(512)`: face un fel de standardizare, pentru a avea date asemanatoare, cumva le aduce intr-o zona mai mica
- `nn.ReLU()`: ma ajuta sa clasific imaginea, de exemplu un scor pozitiv ma ajuta mai mult decat un scor negativ.
- `nn.Dropout(0.3)`: elimina 0.3 din neuroni, ajutand reteaua sa invete mai bine, nu depinde de ce a pierdut
- `nn.Linear(512, 256)`: face la fel, insa mai putin combinatii, aduce un nivel mai mare de generalizare
- `nn.BatchNorm1d(256)`: la fel ca mai sus, dar aplicat pe cele 256 valori
- `nn.Linear(256, num_clase)`: reduc la numarul de clase pe care il am, adica obtin atatea scoruri cate clase am, iar scorul maxim va reprezenta predictia

La augmentari am adaugat `ToTensorV2()` pentru Pytorch, adica transforma in tensori pentru ca reteaua nu poate invata pe imagini de tip numpy.

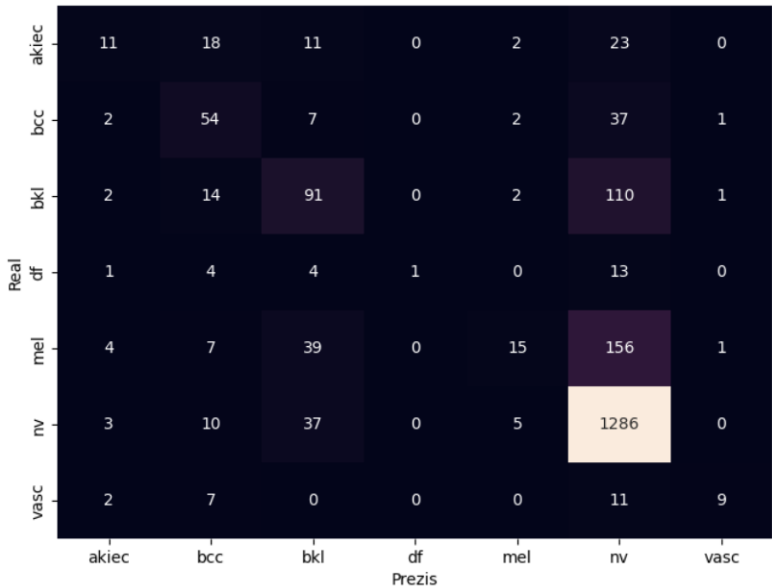
Rezultate obtinute:

1. MLP – Alunite

Am folosit pentru antrenare 40 de epoci, deoarece din grafic modelul inca pare sa invete, deoarece curba inca scade pentru loss si creste pentru acuratete, nu e stabilizata. Initial am testat cu 20 de epoci, dar nu mi s-a parut indeajuns, asa ca am ales sa ma opresc la 40, mi se pare ca am obtinut un rezultat rezonabil. Augmentarile au fost destul de dure.

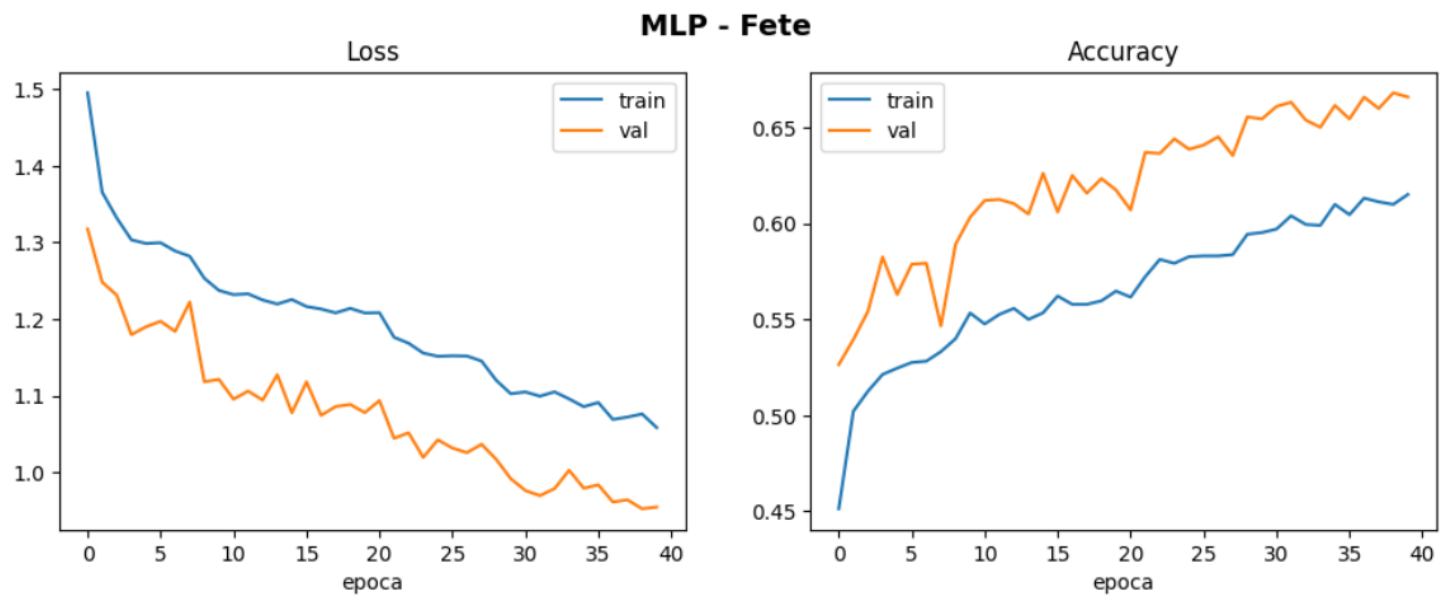


Avem mai jos matricea de confuzie, observam ca domina clasa predominanta, iar cele mai putin folosite sunt neglijate de catre model si tinde sa nu clasifice imaginile in acele categorii. Acest lucru ne arata dezechilibrul dintre clase.

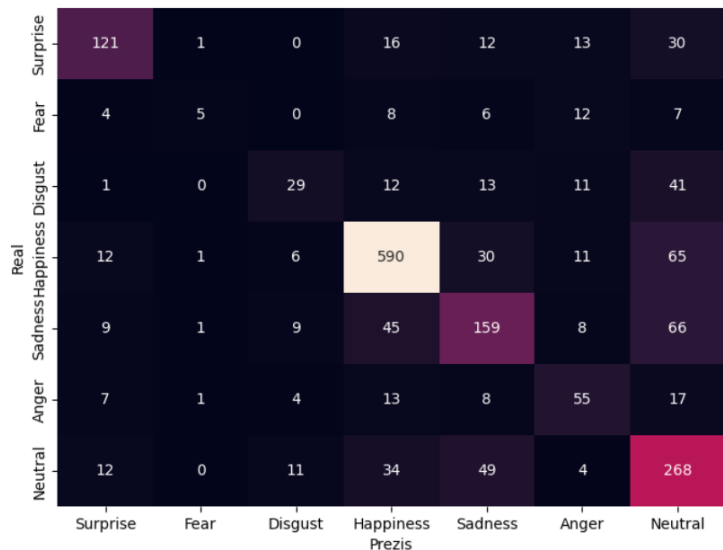


	precision	recall	f1-score	support
akiec	0.44	0.17	0.24	65
bcc	0.47	0.52	0.50	103
bkl	0.48	0.41	0.44	220
df	1.00	0.04	0.08	23
mel	0.58	0.07	0.12	222
nv	0.79	0.96	0.86	1341
vasc	0.75	0.31	0.44	29
accuracy			0.73	2003
macro avg	0.64	0.36	0.38	2003
weighted avg	0.70	0.73	0.68	2003

La fel ca mai sus, insa putem observa ca modelul invata mai greu fata de alunite, adica converge mai greu, subliniind ca aceasta clasificare este mult mai complicata.

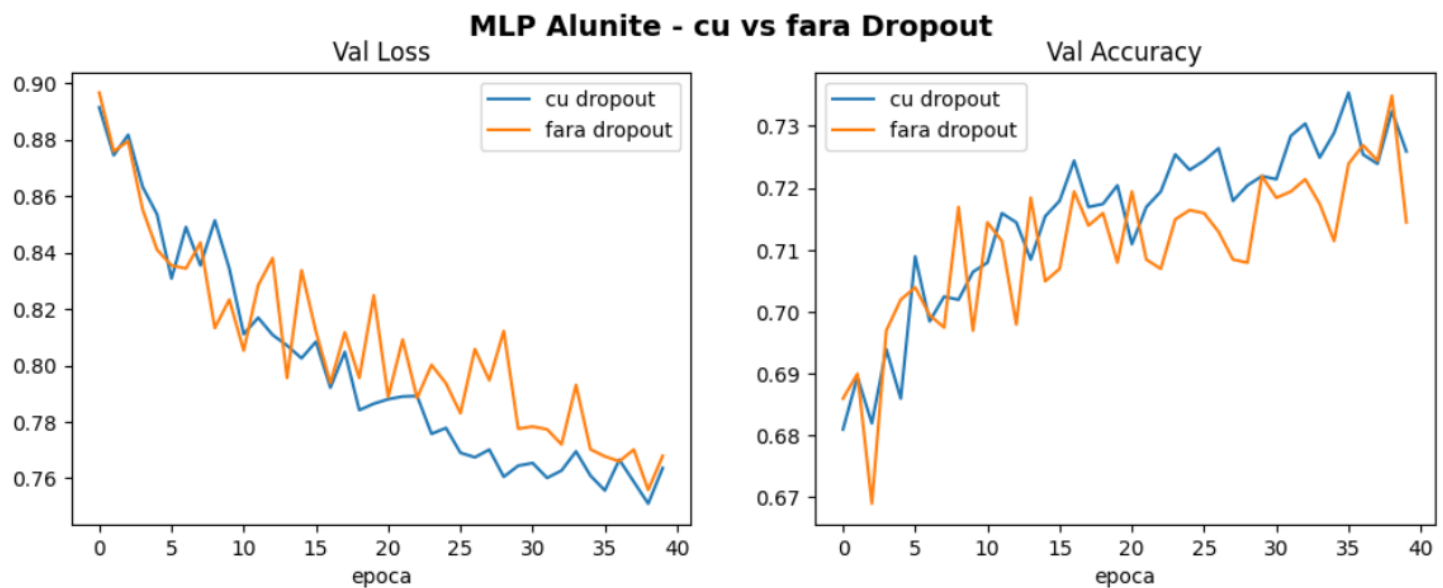


In ceea ce priveste matricea de confuzie si rezultatele obtinute, observam tot ca la alunite, ca unele clase sunt neglijate, insa tinde sa invete cat de cat ok.

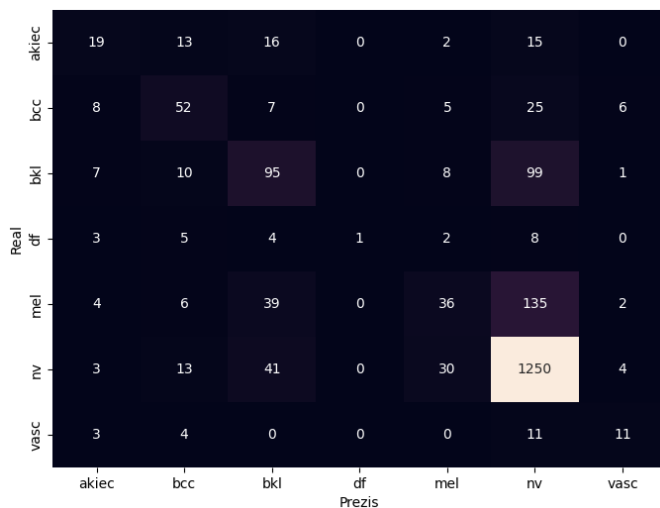


	precision	recall	f1-score	support
Surprise	0.73	0.63	0.67	193
Fear	0.56	0.12	0.20	42
Disgust	0.49	0.27	0.35	107
Happiness	0.82	0.83	0.82	715
Sadness	0.57	0.54	0.55	297
Anger	0.48	0.52	0.50	105
Neutral	0.54	0.71	0.61	378
accuracy			0.67	1837
macro avg	0.60	0.52	0.53	1837
weighted avg	0.67	0.67	0.66	1837

Putem observa mai jos ca la mlp fara dropout situatia scapa putin de sub control, nu mai are aceeasi stabilitate ca mai sus.

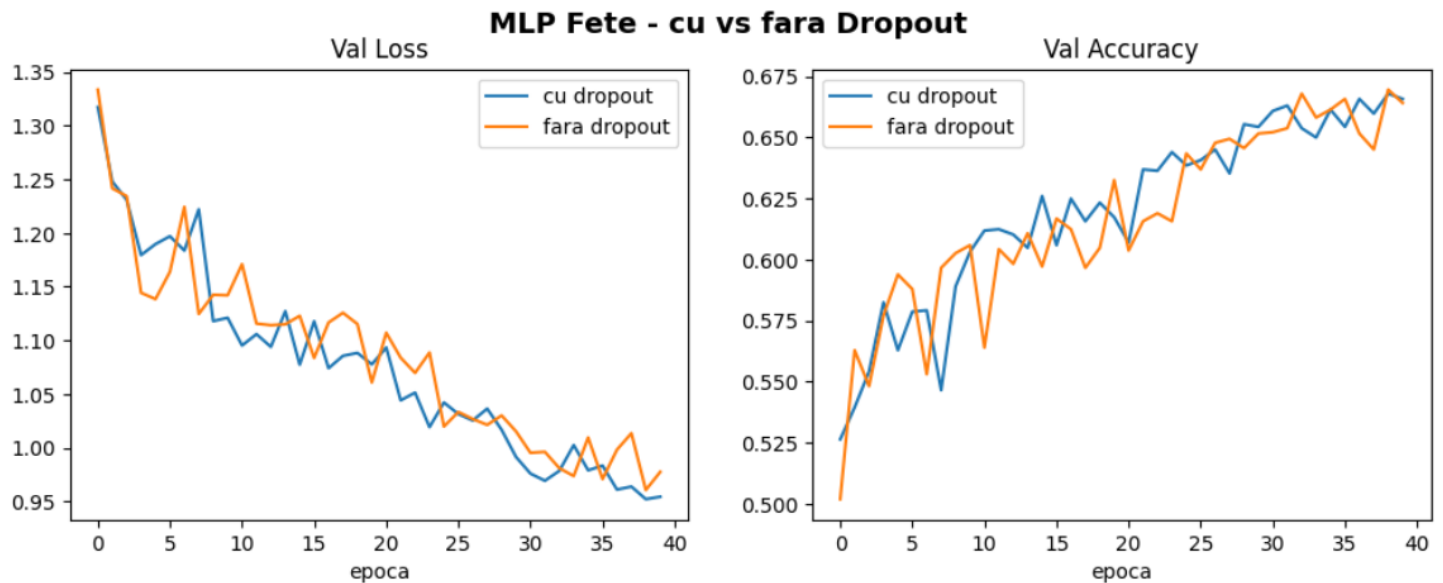


Rezultate similare, observam ca df nu a fost identificata deloc. Deci nu pot spune ca se observa o imbunatatire pentru performanta.

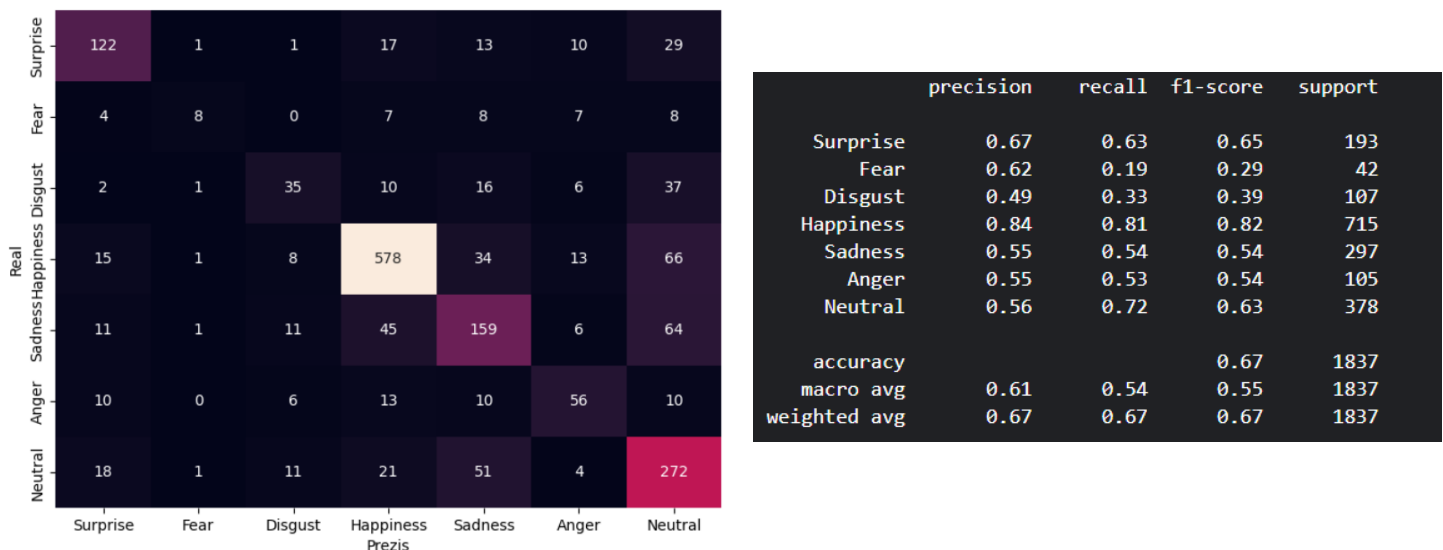


	precision	recall	f1-score	support
akiec	0.40	0.29	0.34	65
bcc	0.50	0.50	0.50	103
bkl	0.47	0.43	0.45	220
df	1.00	0.04	0.08	23
mel	0.43	0.16	0.24	222
nv	0.81	0.93	0.87	1341
vasc	0.46	0.38	0.42	29
accuracy			0.73	2003
macro avg	0.58	0.39	0.41	2003
weighted avg	0.70	0.73	0.70	2003

Observam ca cele 2 curbe, cu si fara dropout, aproape se suprapun, adica fara dropout nici nu imbunatatesc, nici nu scade performanta.



Rezultate asemanatoare exact cum am observant si din grafic.



Tin sa mentionez ca am adaugat aici primul subpunct de la 3.5. Ca si rezultate per total, am obtinut pe Kaggle rezultate bune, care au depasit pragurile. Faptul ca MLP foloseste un vector de pixeli, nestiind ca ei formeaza o imagine intreaga, poate fi un dezavantaj al acestei arhitecturii. La aceasta se adauga si dezechilibrul claselor, am putut observa asta la alunite, unde a invatat mai repede clasa majoritara si ofera o predictie mai mare pentru ea.

In plus, pentru comparatia dropout/fara dropout, nu am observant imbunatatiri relevante. Augmentarile mai dure au fost de ajuns pentru model, inasa daca, compar rezultatele obtinute intre alunite si fete, cele de pe Kaggle, am observat ca mlp a functionat mai bine pentru fete. Un motiv ar fi dezechilibrul claselor, pentru alunite se observa acest lucru destul de mult, in comparatie cu fetele. Un alt motiv ar putea fi ca la alunite e mai greu de identificat, avand in vedere ca pot exista mai multe variatii de situatii, pe cand la fete este mai usor sa identifice o

expresie, deoarece fetele sunt asemanatoare ca structura, ochi, gura, etc. Daca un zambet e asemanator cu altul, la alunate pot face parte din aceeaasi clasa, dar sa arate relative diferit.

3.3 – *Arhitectura CNN*

Diferenta intre CNN si MLP, este ca la CNN reseaua convolutionala stie ca lucreaza cu o imagine, stie ca pixeli formeaza o imagine. Aici CNN stie cum trebuie sa arate puzzle-ul ca imagine la final si trebuie sa potriveasca piesele doar. Face clasificarea pieselor treptat, de la chestii simple, precum margini, culori la chestii mai complexe, precum forme, textura.

Arhitectura CNN foloseste 4 straturi convolutionale si 2 straturi liniare. Am facut aceasta alegere pentru ca 4 straturi convolutionale reusesc sa generalizeze cat mai bine, devin din ce in ce mai abstracte, pentru a identifica forme complexe, iar 2 straturi liniare pentru ca primul restrange spatiul, iar al doilea face clasificarea finala. Am incercat sa nu produc overfitting, de aceea am ramas la aceasta arhitectura.

Am folosit urmatoarele straturi convolutionale:

- *nn.Conv2d*: aplica 32 de filtre pe imagini, fiecare filtru cauta un tipar vizual, iar la final returneaza 32 de harti cu unde a gasit fiecare tipar
- *nn.BatchNorm2d*: cumva restrange valorile, adica sa aiba o scala comuna, un fel de standardizare pentru a putea compara (straturile de normalizare)
- *nn.ReLU*: valoarea negativa devine 0, altfel ramane la fel pentru a putea invata, deoarece s-ar fi comportat ca un singur strat liniar
- *nn.MaxPool2d*: se reduce imaginea la jumate si se pastreaza maximul, asta pentru ca imaginile nu sunt toate centrate, asa cum este si firesc. Iar modelul prin asta invata (straturi de pooling)
- *nn.Conv2d*: al 2-lea strat convolutional, aici are hartile de la primul strat si deja avanseaza sa identifice forme complexe, de la o margine simpla, la o margine mai abrupta. Am dublat numarul de filtre, deoarece sunt mai multe combinatii posibile aici
- *nn.Conv2d*: din nou, face acelasi lucru, intra tot mai adanc si devine tot mai complex
- *nn.AdaptiveAvgPool2d*: calculeaza media fiecărei harti si returneaza o singura valoare. Practic restrange toata informatia din imagine (straturi de pooling)

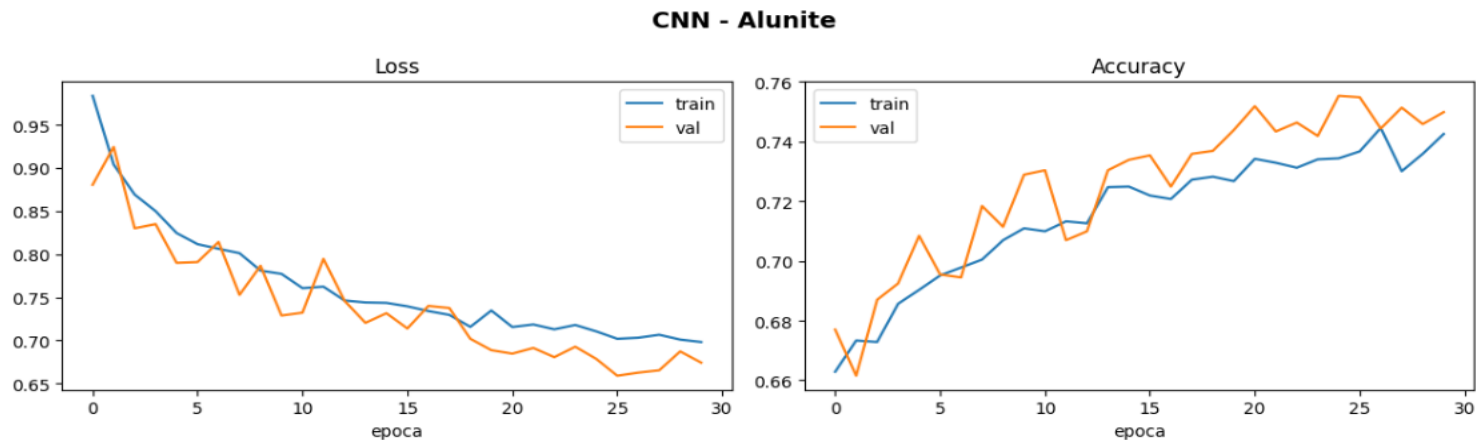
Iar ca straturi liniare am folosit:

- *nn.Flatten*: transforma in formatul pe care straturile liniare il inteleg
- *nn.Linear*: primul strat linear, ia valorile extrase de CNN si le compacteaza pentru a abstractica
- *nn.Dropout*: elimina aleatory 30% din neuroni la fiecare pas (regularizare prin dropout)
- *nn.Linear*: al doilea strat care rezuma totul la numarul de clase

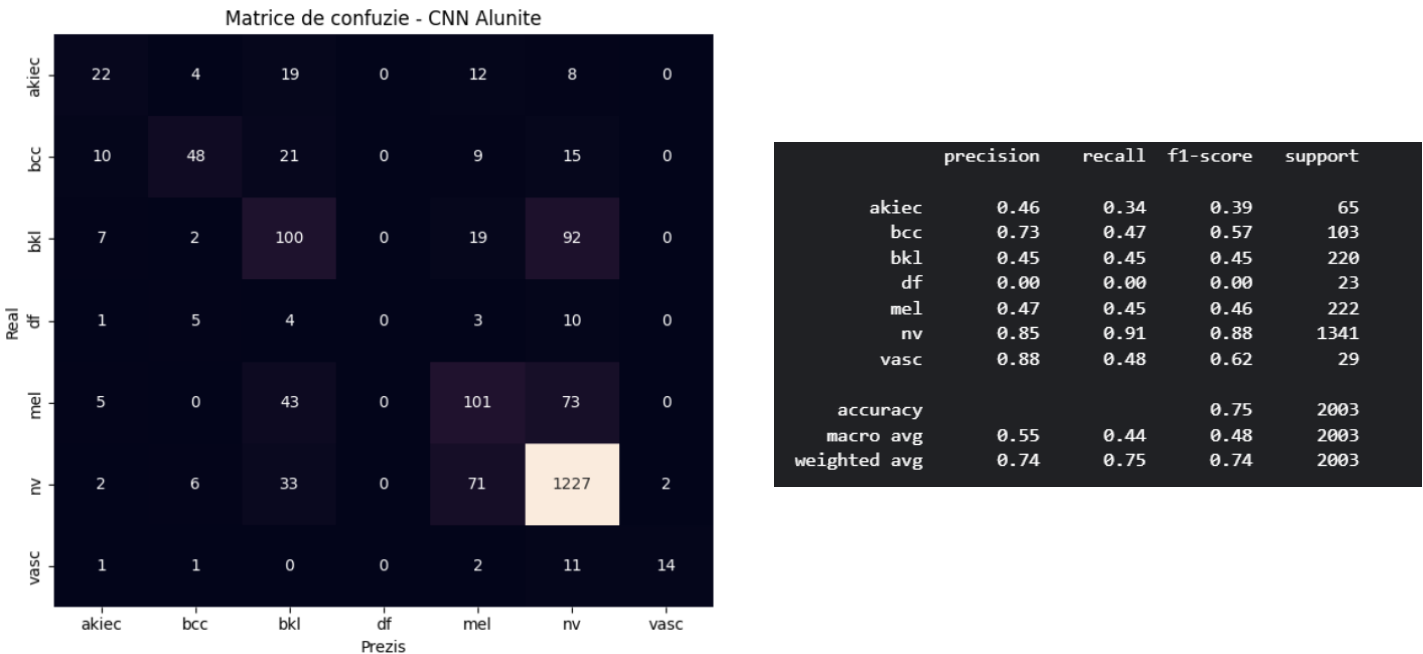
In plus, pentru alunate folosesc dimensiunea 5x5, pentru a vedea mai bine conturul, asimetria etc, insa la fete folosesc 3x3, pentru ca vreau sa observe detaliile expresiei faciale mai indeaproape. Filtrele cresc treptat (32, 64, 128, 256) pentru a deveni mai complex. La

Conv2d, bias = false, pentru ca deja are propriul lui bias si ar fi degeaba. Am ales AdaptiveAvgPool pentru ca in enunt se specifica faptul ca este solutia moderna, dar si pentru ca este mai putin agresiv decat celalalt.

Mai jos, avem graficul pentru CNN alunite, putem observa ca modelul converge stabil, folosind 30 de epoci. Modelul inca pare sa fie in invatare, deci as fi putut sa folosesc mai multe epoci, insa mi-am atins scopul si pragul.

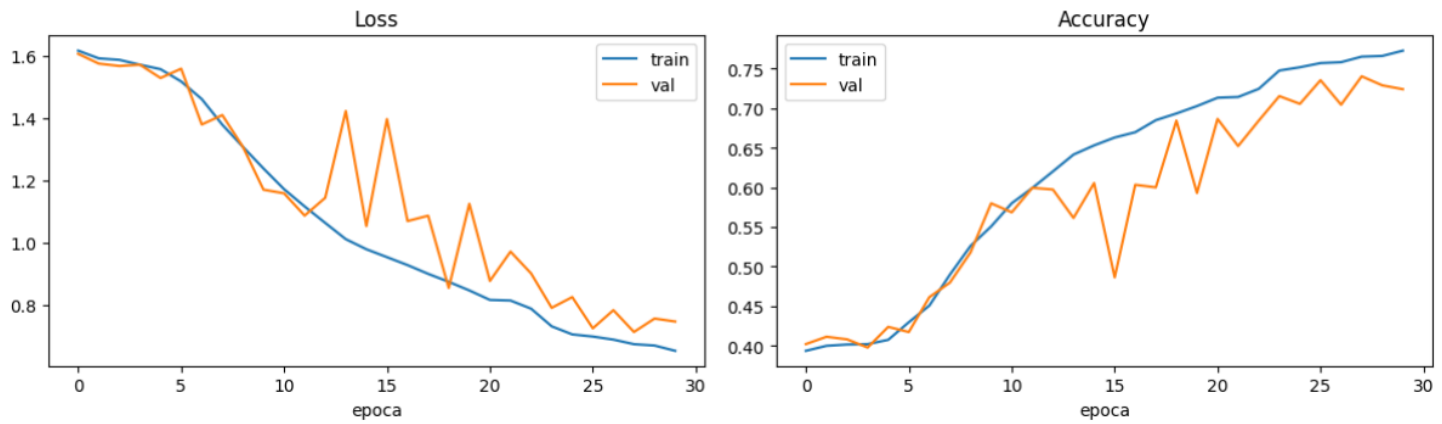


Am afisat si matricea de confuzie, precum si metricile. Observam ca am obtinut acuratete 0.75 si F1 macro 0.48, fiind o imbunatatire fata de MLP. CNN functioneaza mai bine pentru clasele minoritare, inasa ce putem observa este ca df inca nu poate fi detectata de catre model. Deci, dezechilibrul claselor inca este destul de mare si impactant asupra clasei. Bineinteles, prezice majoritar nv fiind clasa predominanta, dar chiar si asa acorda atentie si claselor minoritare.



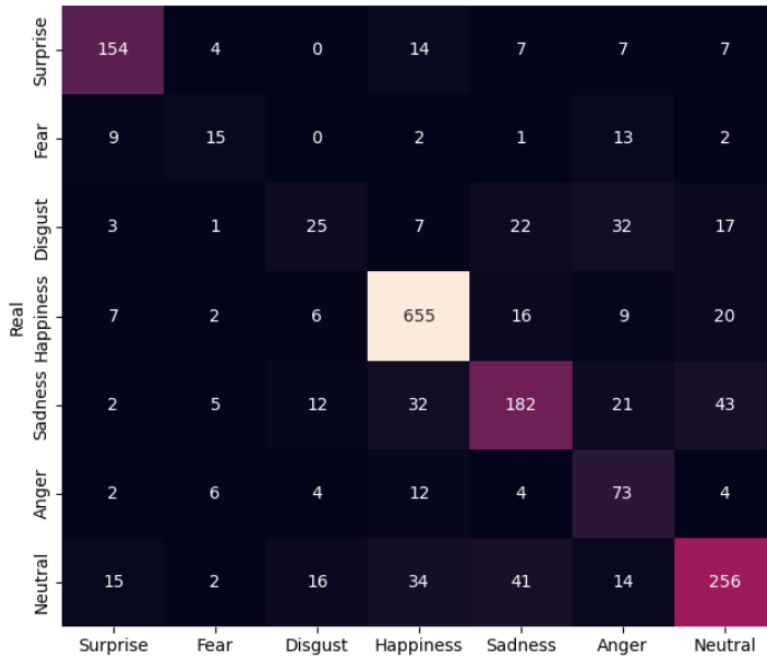
Continuam cu CNN pentru fete, fata de alunite, observam niste variatii anormale intre loss si acuratete. La loss, observam ca urca, inasa la acuratete coboara. Asta ne spune ca exista o instabilitate in timpul antrenarii, poate pentru ca learning rate-ul este prea mare in acele momente. Totusi, de pe la epoca 20 incepe sa converge si se stabilizeaza.

CNN - Fete



Am obtinut o acuratete egale cu 0.74 si f1 macro egal cu 0.61. Se poate vedea si aici imbunatatirea fata de mlp. Happiness ramane clasa majoritara si cea mai des prezisa. Totusi, restul claselor nu sunt neglijate, inasa observam o confuzie intre Disgust si Sadness, precum si intre Disgust si Anger.

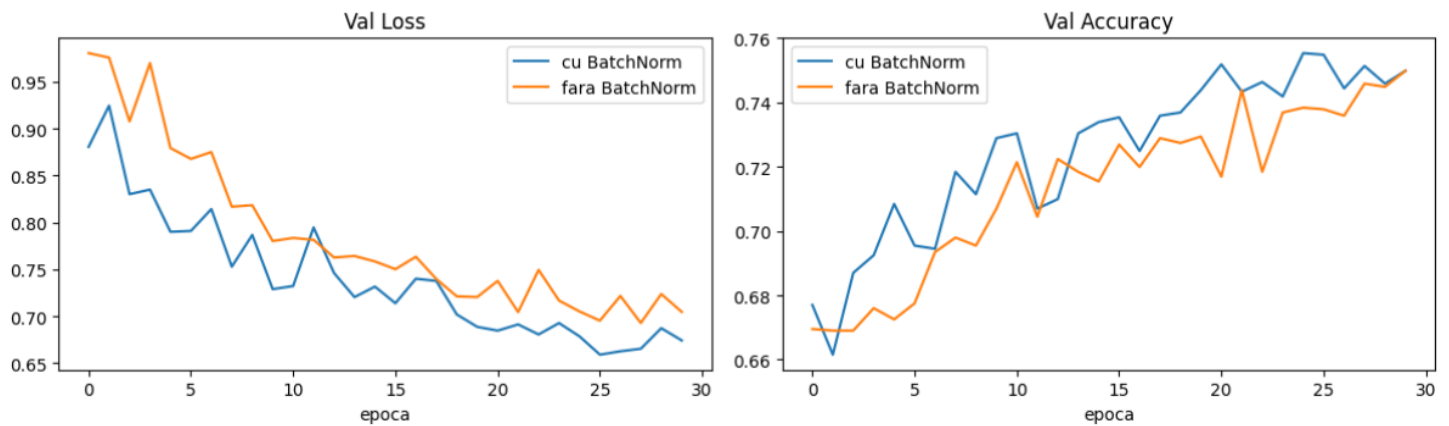
Matrice de confuzie - CNN Fete



	precision	recall	f1-score	support
Surprise	0.80	0.80	0.80	193
Fear	0.43	0.36	0.39	42
Disgust	0.40	0.23	0.29	107
Happiness	0.87	0.92	0.89	715
Sadness	0.67	0.61	0.64	297
Anger	0.43	0.70	0.53	105
Neutral	0.73	0.68	0.70	378
accuracy			0.74	1837
macro avg	0.62	0.61	0.61	1837
weighted avg	0.74	0.74	0.74	1837

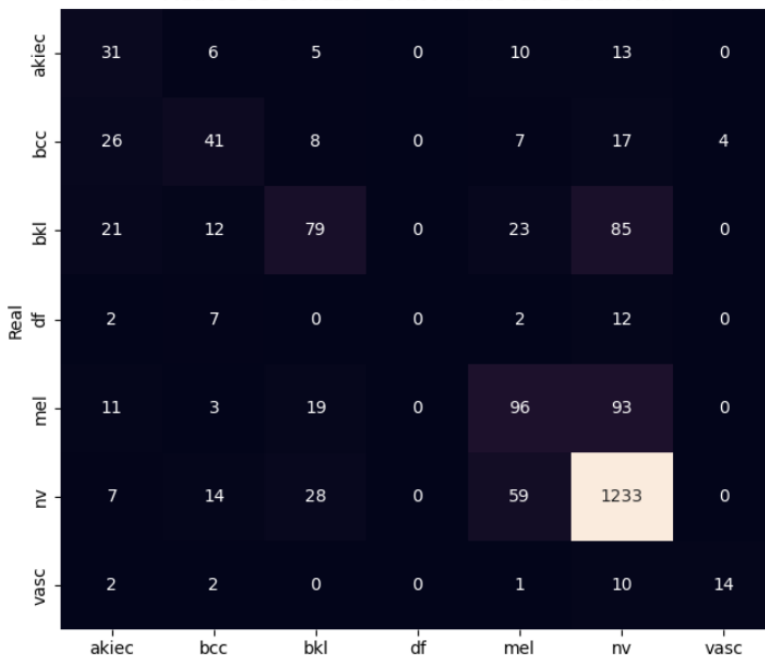
Urmeaza comparatia de la 3.5 cu/fara BatchNorm. Din grafic observam ca obtinem un loss mai mic cand folosesc BatchNorm si converge mai repede. Fara BatchNorm ar avea nevoie de mai multe epoci pentru a ajunge la aceleasi rezultate. Dar spre final, se observa ca diferenta este foarte mica, deci nu observam ceva foarte impresionant si diferit, ceva impactant.

Ablatie CNN Alunite - cu vs fara BatchNorm



Am obtinut acuratete 0.75 si f1 macro 0.45, mai putin decat la varianta cu BatchNorm. Clasele minoritate sunt mai rar prezise, insa nu scade cu mult procentul. Df inca nu este remarcata. Asadar, BatchNorm converge mai repede la inceput si are o performanta mai buna. Ajuta la stabilizare si reduce oscilatiile, desi rezultatele nu sunt foarte diferite. Totusi, se poate observa aceasta mica diferenta care in cele din urma cu siguranta conteaza.

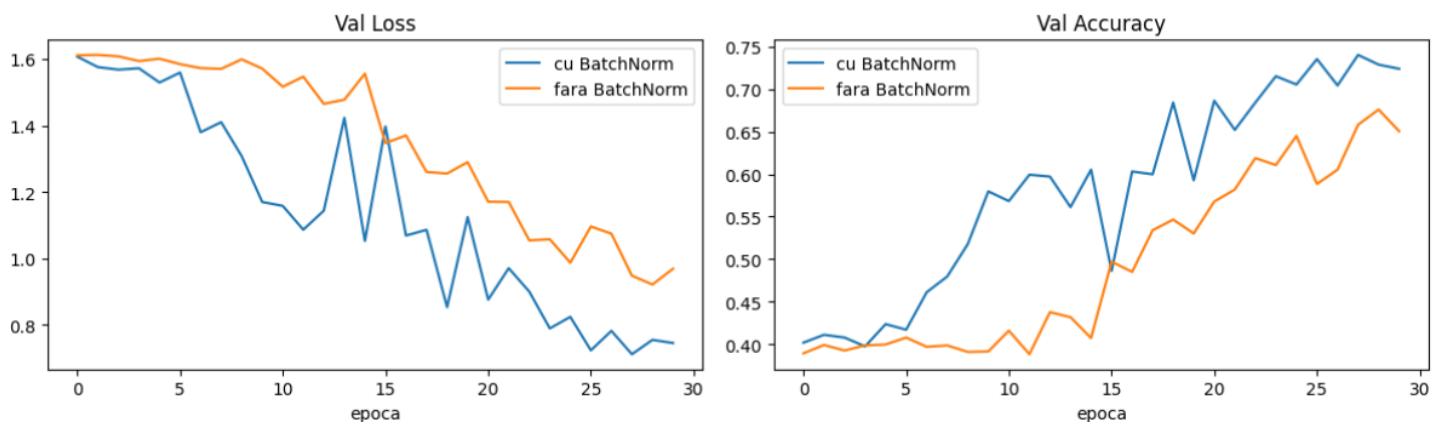
Matrice de confuzie - CNN Alunite fara BatchNorm



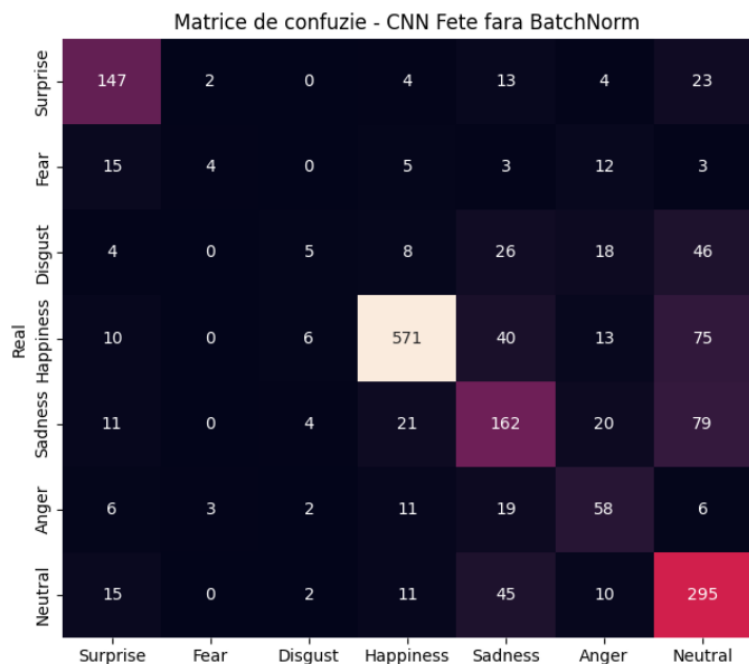
	precision	recall	f1-score	support
akiec	0.31	0.48	0.38	65
bcc	0.48	0.40	0.44	103
bkl	0.57	0.36	0.44	220
df	0.00	0.00	0.00	23
mel	0.48	0.43	0.46	222
nv	0.84	0.92	0.88	1341
vasc	0.78	0.48	0.60	29
accuracy			0.75	2003
macro avg	0.50	0.44	0.45	2003
weighted avg	0.73	0.75	0.73	2003

Din graficul de mai jos, observam ca fara BatchNorm, la fel ca la alunite, converge mai lent, dar atinge o valoare mai mare pentru loss (aproximativ 0.9) fata de varianta cu BatchNorm unde obtine aproximativ 0.70 dupa 30 de epoci. La acuratete, obtine pentru modelul fara BatchNorm aproximativ 0.65, iar cu BatchNorm 0.73. Totusi, curba cu BatchNorm e mai curata, fara multe oscilatii, adica are o antrenare mai stabila. Chiar si asa, modelul cu BatchNorm recupereaza spre final si ajunge sa aiba o performanta mai buna.

Ablatie CNN Fete - cu vs fara BatchNorm



Din matricea de confuzie, observam ca modelul clasifica bine clasele majoritare, dar pe cele minoritare le neglijeaza, de exemplu Disgust are doar 5 corecte. Iar ca acuratete si f1-score, observam ca pe clasele minoritare avem un scor mic, ceea ce am vazut si in matricea de confuzie. Pentru clasele majoritare, obtine scor mai mare. Astfel, varianta cu BatchNorm aduce un plus pentru performanta, dar si pentru viteza de convergenta. Modelul fara BatchNorm ne-ar incetini, dar si distruge invatarea modelului pe clase minoritare.

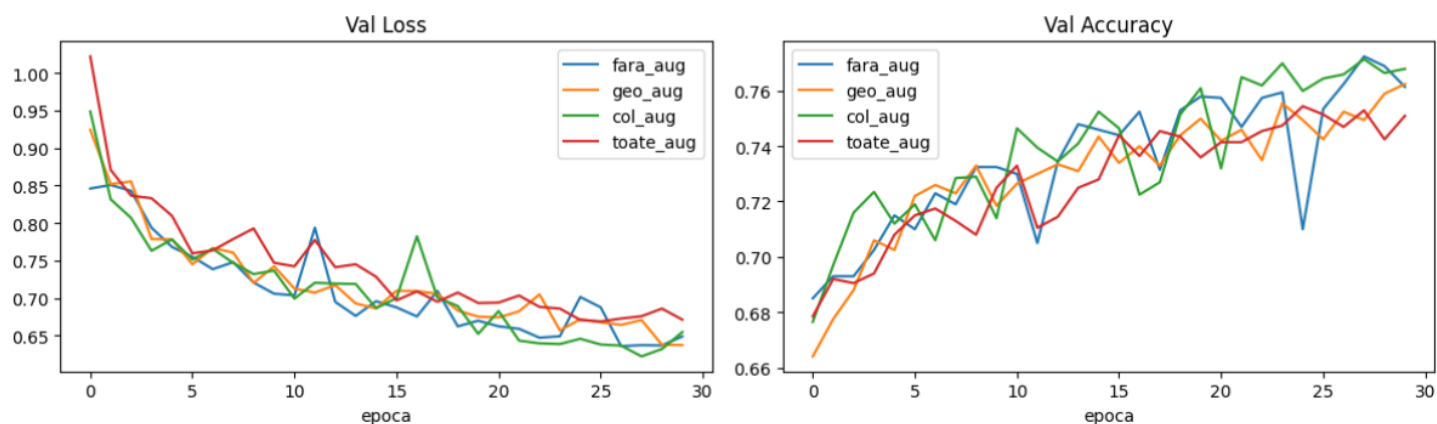


	precision	recall	f1-score	support
Surprise	0.71	0.76	0.73	193
Fear	0.44	0.10	0.16	42
Disgust	0.26	0.05	0.08	107
Happiness	0.90	0.80	0.85	715
Sadness	0.53	0.55	0.54	297
Anger	0.43	0.55	0.48	105
Neutral	0.56	0.78	0.65	378
accuracy			0.68	1837
macro avg	0.55	0.51	0.50	1837
weighted avg	0.68	0.68	0.66	1837

Acum ajungem la toate configuratiile de augmentare: fara augmentari, cu augmentari geometrice, cu augmentari in spatiul de culoare, cu toate augmentarile.

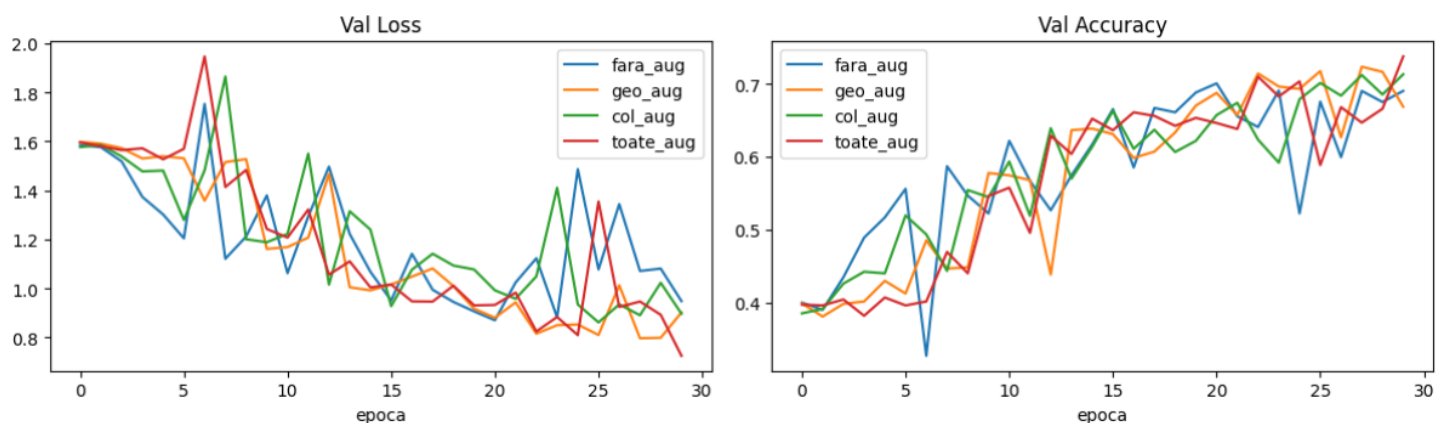
Pentru alunite, putem vedea ca avem o variatie intre rezultatele obtinute. Observam oscilatii pentru modelul fara augmentari si pentru cel in spatiul de culoare, dar spre final se stabilizeaza. Pentru culoare era de asteptat, avand in vedere ca la alunite culoarea este foarte importanta pentru clasificare. Pentru cele geometrice, au un parcurs relativ smooth, la fel si pentru toate augmentarile.

Ablatie CNN Alunite - Augmentari



Pentru fete, putem observa ca toate variantele sunt asemanatoare, converg aproximativ la fel. Augmentarile geometrice si augmentarile in spatiul de culoare par sa stabilizeze curba, nu am atat de multe oscilatii. La toate augmentarile, observam ca parcursul este un pic mai dur la inceput, dar recupereaza rapid spre final. Pentru cazul cu fara augmentari, observam o oscilatie mai grava in jurul epocii 6, iar acest lucru ne arata o instabilitate, Acestea nu ofera o performanta wow, avand in vedere ca setul de date este deja destul de variat.

Ablatie CNN Fete - Augmentari



Per total, combinarea tuturor nu produce neaparat un rezultat foarte bun, iar acest lucru ne spune ca desi ii oferim mai mult modelului acesta nu reuseste sa progreseze, ci e un process invers, modelul devine mai prost.

3.5 – cu/fara dezechilibru

Am observant pana acum ca ambele seturi de date prezinta un dezechilibru in ceea ce priveste clasele. In setul de date pentru alunite am observat ca si clasa majoritara nv, iar minoritara df. In setul de date pentru fete am observat ca si clasa majoritara Happiness, iar minoritara Fear.

Din cauza dezechilibrului, modelul tinde sa prezica clasa majoritara mai des si sa negligeze clasele minoritare, obtinand astfel o acuratete mare doar prin prezicerea clasei majoritare. De exemplu, o astfel de eroare este critica in medicina, asa cum avem cazul cu alunite. Un diagnostic gresit poate pune in pericol viata pacientului.

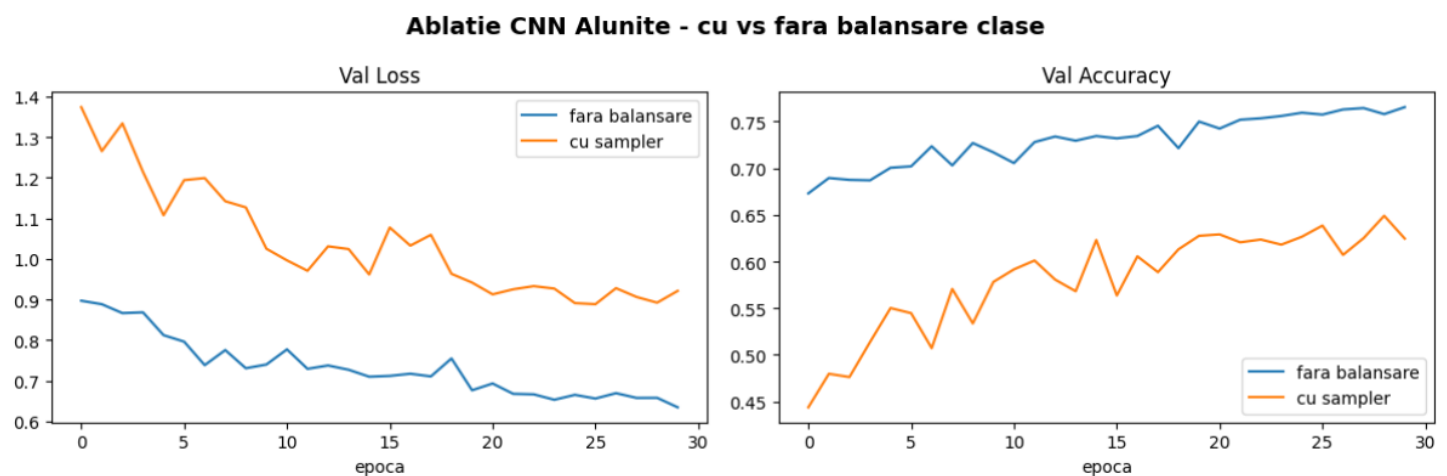
Pentru a repara acest dezchilibru am folosit `WeightedRandomSampler` care atribuie fiecarui exemplu o pondere invers proportionala cu frecventa clasei sale. Astfel, la fiecare epoca, clasele rare sunt utilizate mai frecvent, rezultand un echilibru in prezicere fara sa modificam datele initiale. Am pastrat arhitectura de la CNN si aceeasi hiperparametri.

Am antrenat doua variante de arhitecturii CNN pentru fiecare set de date, diferenta fiind reprezentata de catre modul de esantionare a datelor de antrenare. Varianta fara echilibrare foloseste un `DataLoader` standard cu `shuffle = true`, adica fiecare imagine are aceeasi probabilitate de a fi selectata intr-un batch. Astfel clasele majoritare domina inevitabil. Varianta cu echilibru foloseste `WeightRandomSampler`, unde fiecare imagine primeste o pondere calculate ca $w_c = 1 / N_c$, unde N_c e numarul total de exemple din clasa c . Astfel, imaginile rare sunt extrase mai frecvent in fiecare epoca, producand batch-uri mai echilibrate. Cum `replacement = true`, exemplele din clasele minoritare pot aparea de mai multe ori pe parcursul unei epoci.

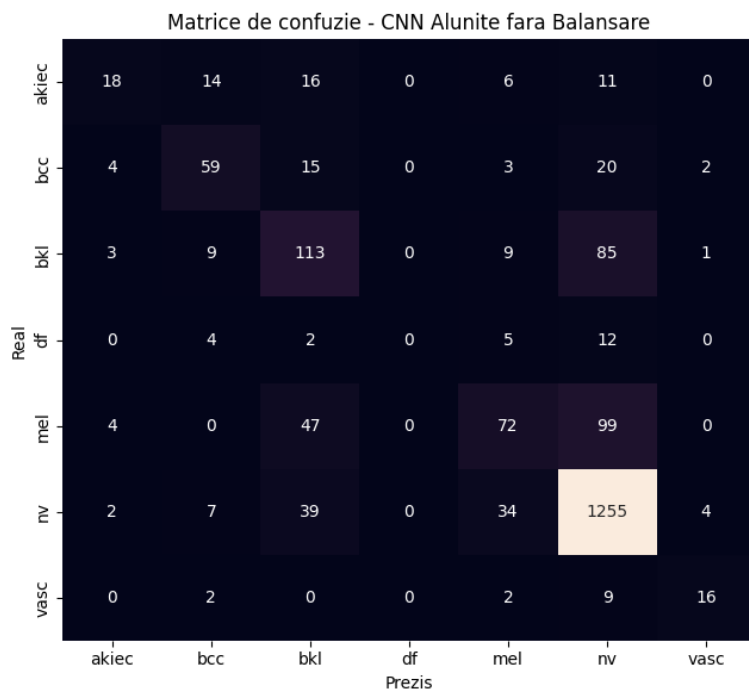
Ambele variante folosesc:

- acelasi CNN cu Batchnorm si 4 straturi convolutionale
- Optimizer Adam cu `lr == 0.001` si `weight_decay = 0.0001`
- Scheduler `ReduceLROnPlateau` cu `patience = 3` si `factor = 0.5`
- 30 de epoci pentru antrenare
- `CrossEntropyLoss` standard

Mai jos putem observa graficul, unde vedem ca modelul fara echilibru obtine un loss mult mai mic, dar acuratete mai mare fata de varianta cu echilibru unde loss-ul este mai mare si acuratetea mai mica. Desi pare ca modelul fara echilibru da mai bine, acest lucru este cat se poate de fals si reuseste sa ne pacaleasca, deoarece acuratetea este acaparata de clasa dominanta `nv` ce reprezinta ea insasi 67% din date.

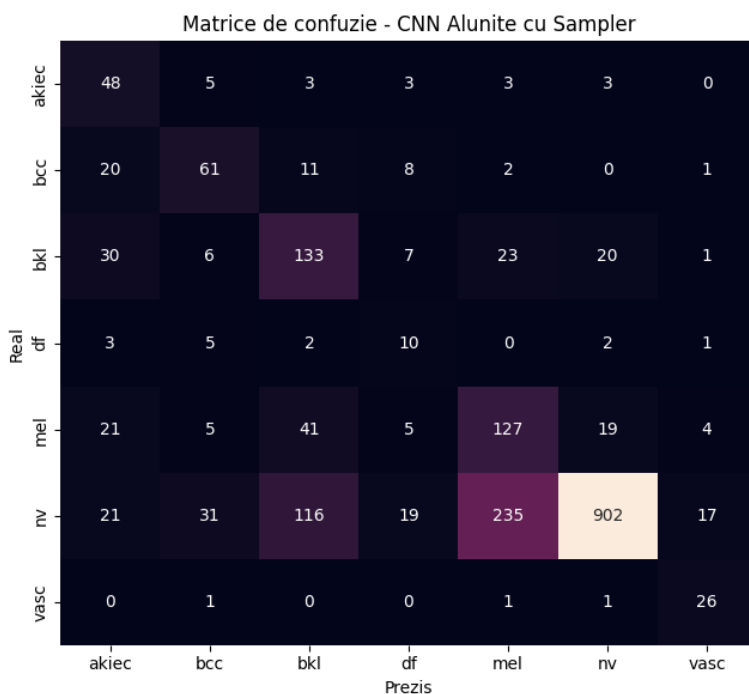


In matricea de confuzie putem observa diferentele clare. Vedem cum clasa majoritata este prezisa mai des, iar clasele minoritare sunt neglijate. Df nu este detectata niciodata, iar restul obtin scoruri foarte mici.



	precision	recall	f1-score	support
akiec	0.58	0.28	0.38	65
bcc	0.62	0.57	0.60	103
bkl	0.49	0.51	0.50	220
df	0.00	0.00	0.00	23
mel	0.55	0.32	0.41	222
nv	0.84	0.94	0.89	1341
vasc	0.70	0.55	0.62	29
accuracy			0.77	2003
macro avg	0.54	0.45	0.48	2003
weighted avg	0.74	0.77	0.75	2003

In schimb, odata cu echilibrul putem observa cum lucrurile se schimba si modelul reuseste sa detecteze df de 10 ori, iar nv este prezis de mai putine ori.

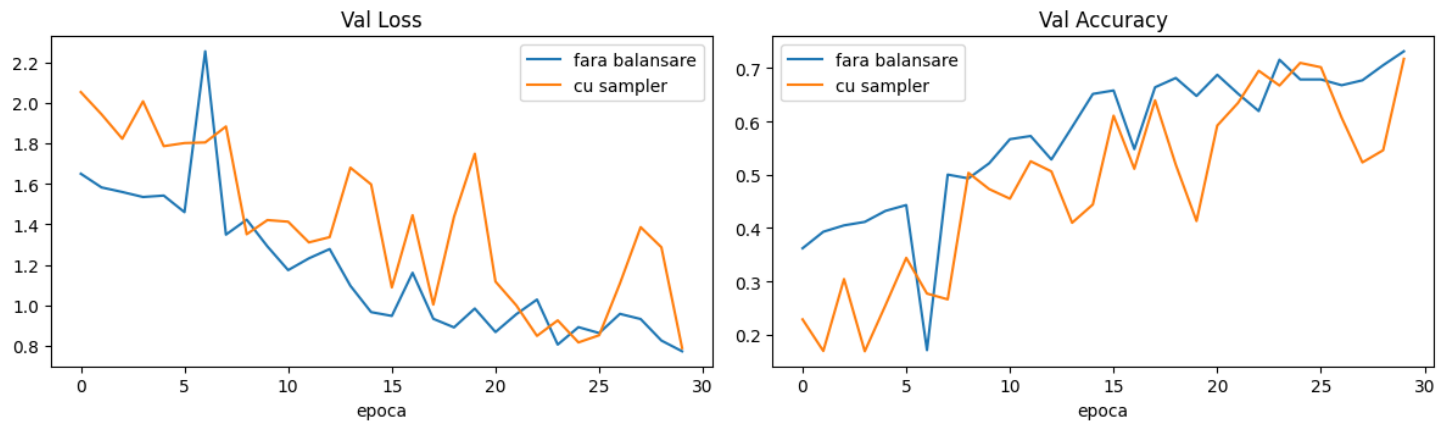


	precision	recall	f1-score	support
akiec	0.34	0.74	0.46	65
bcc	0.54	0.59	0.56	103
bkl	0.43	0.60	0.51	220
df	0.19	0.43	0.27	23
mel	0.32	0.57	0.41	222
nv	0.95	0.67	0.79	1341
vasc	0.52	0.90	0.66	29
accuracy			0.65	2003
macro avg	0.47	0.64	0.52	2003
weighted avg	0.77	0.65	0.69	2003

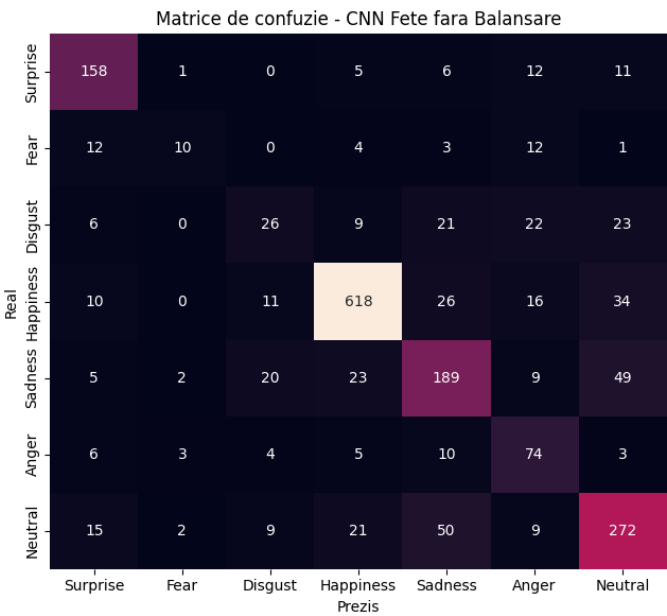
Astfel, echilibrul de clase prin WeightedRandomSampler aduce o imbunatatire, rezolvand problema prezicerii claselor minoritare. Desi acuratetea scade, ne dam seama ca aceasta poate fi foarte inselatoare pe seturi de date dezechilibrate.

Pentru fete observam ca avem curbe asemanatoare in ambele cazuri. Observam oscilatii, iar ce putem spune este ca daca fortam echilibrarea claselor, destabilizeaza antrenarea initiala. Spre final, cele doua curbe se apropie, modelul fara echilibrare ramane mai stabil.

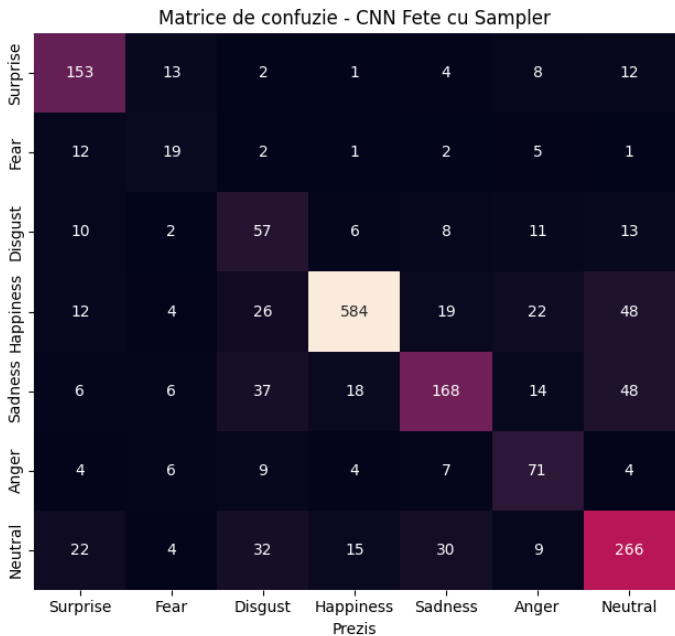
Ablatie CNN Fete - cu vs fara balansare clase



Observam in matricea de confuzie diferenta reala intre cele 2 variante. Observam cum clasele rare ies in evidenta, dar clasele majoritare pierd din atentie.



	precision	recall	f1-score	support
Surprise	0.75	0.82	0.78	193
Fear	0.56	0.24	0.33	42
Disgust	0.37	0.24	0.29	107
Happiness	0.90	0.86	0.88	715
Sadness	0.62	0.64	0.63	297
Anger	0.48	0.70	0.57	105
Neutral	0.69	0.72	0.71	378
accuracy			0.73	1837
macro avg	0.62	0.60	0.60	1837
weighted avg	0.73	0.73	0.73	1837



	precision	recall	f1-score	support
Surprise	0.70	0.79	0.74	193
Fear	0.35	0.45	0.40	42
Disgust	0.35	0.53	0.42	107
Happiness	0.93	0.82	0.87	715
Sadness	0.71	0.57	0.63	297
Anger	0.51	0.68	0.58	105
Neutral	0.68	0.70	0.69	378
accuracy			0.72	1837
macro avg	0.60	0.65	0.62	1837
weighted avg	0.75	0.72	0.73	1837

In concluzie, echilibrarea claselor aduce o imbunatatire pentru clasele minoritare. Spre deosebire de alunite, unde clasele rare sunt critice din punct de vedere medical, pe fete impactul este mai putin important, dezechilibrul este mai moderat, iar modelul fara echilibrare reuseste deja sa recunoasca partial clasele rare.

3.4 – *Finetuning*

Deoarece sertul nostru de date este mic, reseaua noastra este predispusa la riscul de a nu invata suficiente generalizari, sa duca la overfit sau la o performanta mai slaba. De aceea folosim fine-tuning, in loc sa initializam reseaua cu ponderi aleatoare, pornim deja de la un model antrenat pe un set de date mai mare si il adaptam la situatia noastra.

ResNet-18 are 18 straturi, antrenat pe ImageNet (dataset cu 1.2 milioane de iamgini si 1000 de clase). Tocmai pentru ca are o antrenare extinsa, straturile convolutionale stiu sa extraga caracteristici vizuale generale si robuste. Astfel, ar trebui sa fie capabil sa identifice expresii faciale, sa clasifice leziunile pentru alunite.

Noi avem 7 clase, iar ultimul strat fully-connected l-am inlocuit cu un singur strat linear pentru alunite, iar pentru fete un strat Dropout urmat de Linear pentru regularizare suplimentara, deoarece avem o variabilitate mai mare la expresii faciale.

Imaginile le-am redimensionat la 224x224 pixeli si normalizate cu statisticile ImageNet. Am realizat in continuare mai multe experimente.

1. Backbone inghetat complet: toti parametrii ResNet-18, cu exceptia capului de clasificare, sunt inghetati, adica gradientii nu sunt calculate si ponderile nu se modifica. Singurii parametrii antrenabili sunt cei ai stratului fc.
2. Backbone partial dezghetat: straturile 3 si 4 ale ResNet-18-ului sunt dezghetate, aceste straturi fiind cele mai profunde. Au un learning rate mai mic (0.0001) fata de capul nou (0.001). Straturile 1 si 2 sunt inghetate, acestea extrag detalii generale, precum margini, texture. Astfel, reseaua se poate adapta la un nivel mai inalt, fara a distruge caracteristicile de baza deja invatate.

La inceputul fine-tuningului, capul de clasificare este initializat aleator, producand gradient mari si zgomotosi care pot pertuba ponderile pre-antrenate ale backbone-ului. Pentru a preveni acest lucru, am folosit warmup linear: in primele 3 epoci, learning rate-ul creste liniar de la 0 la valoarea tinta, dupa care urmeaza cosine annealing care il scade progresiv pana la finalul celor 40 de epoci. Variant afara warmup foloseste direct cosine annealing de la prima epoca.

Functia de loss folosita e CrossEntropyLoss cu ponderi per clasa calculate invers proportional cu frecventa fiecarei clase. Am aplicat si label smoothing (0.05 pentru alunite, 0.1 pentru fete), care regularizeaza, prevenind supraincercarea modelului cand prezice.

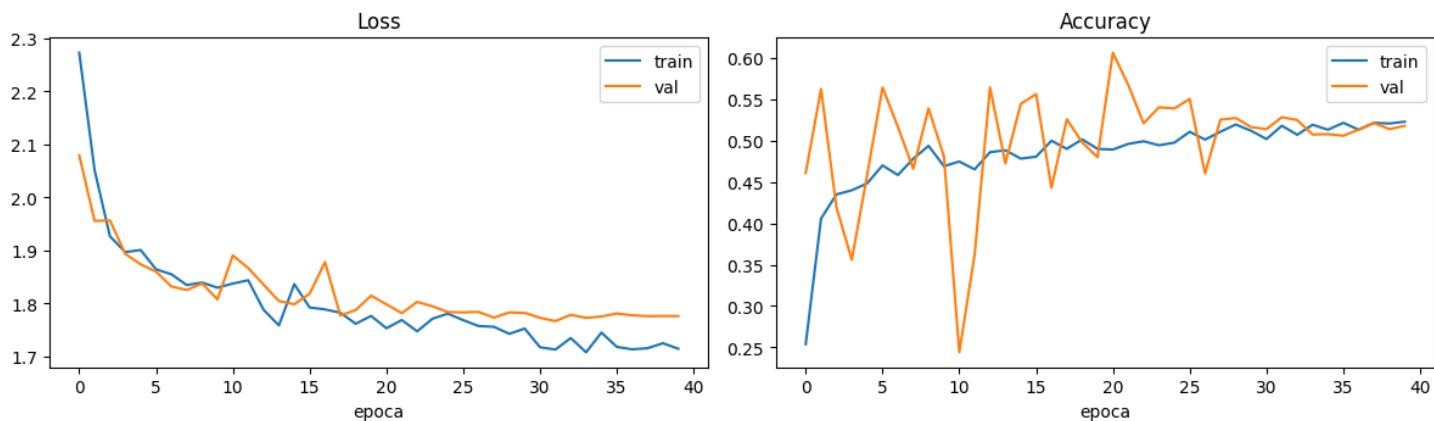
In total am antrenat 8 modele, cate 4 per dataset:

- Frozen + warmup
- Frozen + fara warmup
- Partial frozen + warmup
- Partial frozen + fara warmup

Toate au fost antrenate cu 40 de epoci, iar cel mai bun rezultat a fost selectat dupa f1-macro.

Acum le voi lua pe fiecare in parte pentru a vedea ce am obtinut de fapt si de drept. Mai jos, putem observa graficul pentru alunite frozen cu warmup. Curba loss converge similar atat pe train, cat si pe validare, nu observam fenomenul de overfit, converge bine. In schimb, pentru acuratete, observam oscilatii majore pe validare, mai ales la epoca 10, dar reuseste sa se stabilizeze spre final. Aceste oscilatii apar deoarece acolo capul de clasificare se calibreaza.

ResNet18 Alunite - frozen cu warmup



In ceea ce priveste matricea de confuzie si rezultatele obtinute pentru metrici, am obtinut o acuratete 0.61 si f1 0.43. Modelul reuseste sa prezica pana si clasa minoritara df, desi exista o variatie intre clasa majoritara nv 0.77 si df 0.21. Insa observam la df este diferenta dintre recall (0.61) si precision (0.12), acest lucru ne spune ca modelul le prezice frecvent si gresit. Putem vedea in matricea de confuzie false pozitivele (138 mel, 98 bkl, 78 df prezise gresit ca nv).

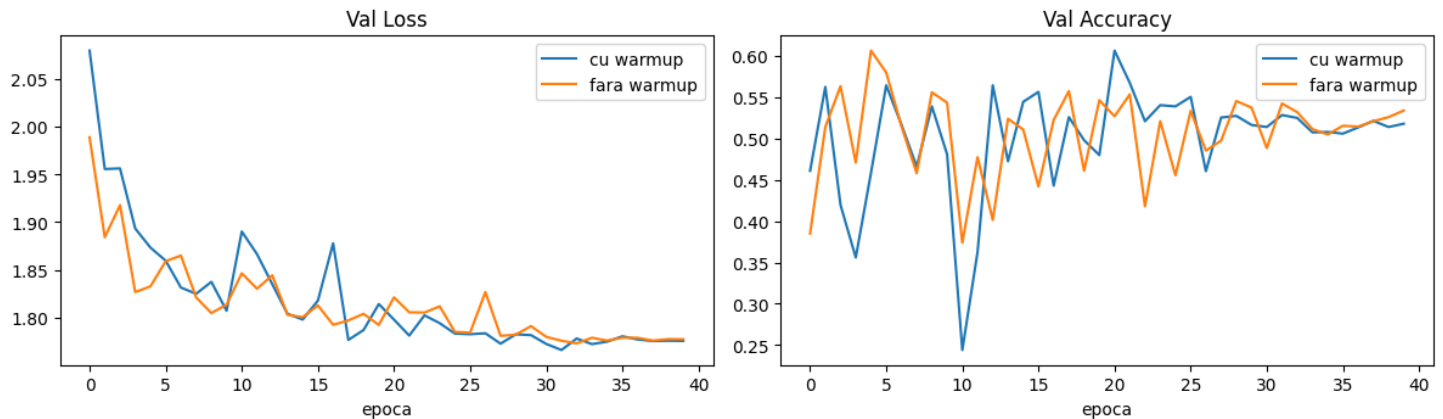
Matrice de confuzie - Alunite frozen cu warmup

	akiec	bcc	bkl	df	mel	nv	vasc
akiec	44	7	9	1	2	2	0
bcc	20	53	15	3	4	1	7
bkl	29	13	127	9	14	21	7
df	1	1	5	14	1	0	1
mel	36	9	45	6	93	17	16
nv	48	27	98	78	138	858	94
vasc	0	2	0	2	0	0	25

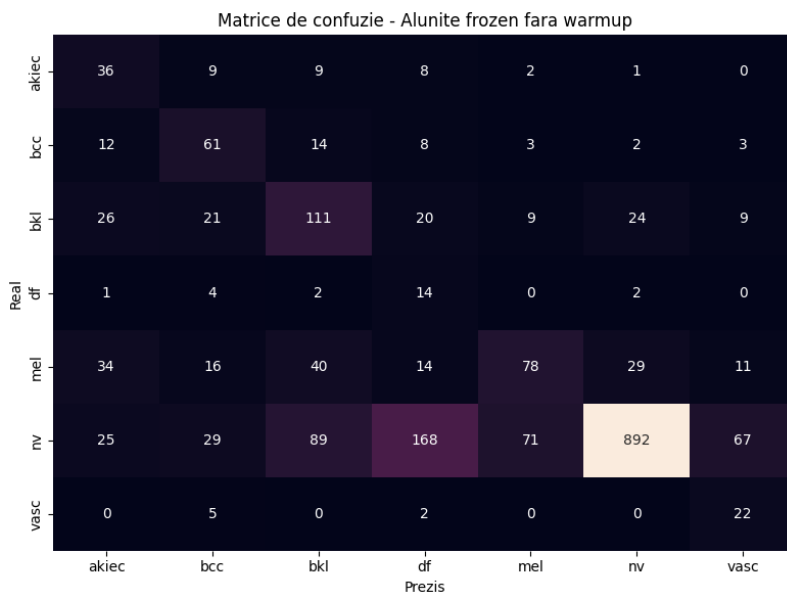
	precision	recall	f1-score	support
akiec	0.25	0.68	0.36	65
bcc	0.47	0.51	0.49	103
bkl	0.42	0.58	0.49	220
df	0.12	0.61	0.21	23
mel	0.37	0.42	0.39	222
nv	0.95	0.64	0.77	1341
vasc	0.17	0.86	0.28	29
accuracy			0.61	2003
macro avg	0.39	0.61	0.43	2003
weighted avg	0.76	0.61	0.65	2003

Mai jos observam cum amandoua converg similar. Varianta fara warmup porneste cu un loss mai mic, ceea ce ne arata learning rate-ul aplicat de la inceput. Ambele prezinta oscilatii pe acuratete in prima parte, urmand sa se stabilizeze spre final. Diferenta este infima spre final, ceea ce ne arata ca warmup nu aduce o imbunatatire extraordinara.

ResNet18 Alunite - frozen fara warmup - cu vs fara Warmup



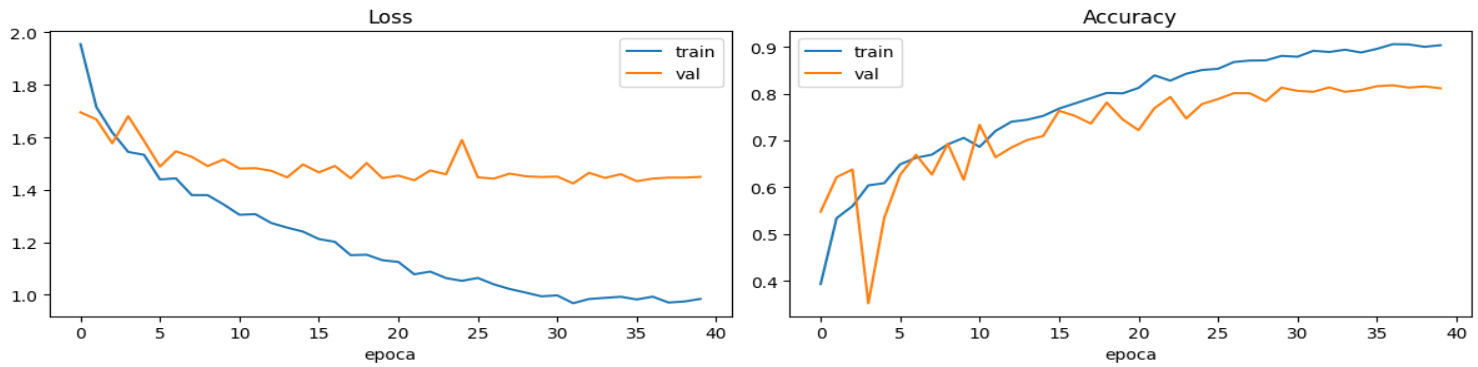
Rezultatele obtinute ne arata ca fata de varianta cu warmup, cea fara warmup distribuie mai uniform erorile, nv are 892 corecte, genereaza mai putine false positive, de exemplu pentru df are 168 fata de 78. Df ramane in continuare cea mai grea clasa de prezis. Asadar, warmup-ul nu aduce o diferenta semnificativa.



	precision	recall	f1-score	support
akiec	0.27	0.55	0.36	65
bcc	0.42	0.59	0.49	103
bkl	0.42	0.50	0.46	220
df	0.06	0.61	0.11	23
mel	0.48	0.35	0.41	222
nv	0.94	0.67	0.78	1341
vasc	0.20	0.76	0.31	29
accuracy			0.61	2003
macro avg	0.40	0.58	0.42	2003
weighted avg	0.76	0.61	0.66	2003

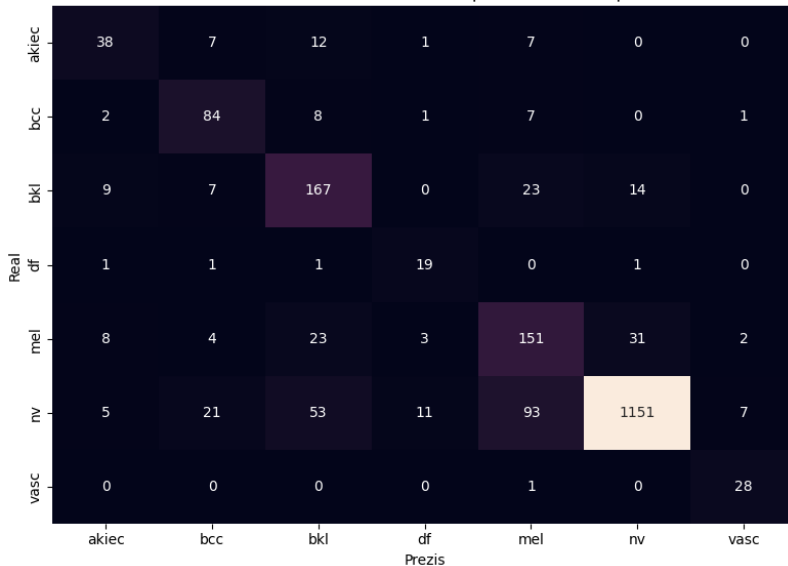
Am obtinut acum cea mai buna configuratie de pana acum. Varianta partial unfreeze cu warmup. Train loss-ul este constant, pe validare se stabilizeaza mai sus dupa epoca 10 si ramane acolo pana la final, asta ne spune ca avem un overfit prezent. Invata pe train, dar nu mai generalizeaza pe validare. La acuratete, vedem tot un parcurs lin, modelul a invatat reprezentarile utile. Oscilatia de la inceput este caracteristica fazei de warmup cand learning rate inca e mic si modelul se recalibreaza.

ResNet18 Alunite - partial unfreeze cu warmup



Observam rezultate mult mai bune, de la 0.42 am crescut la 0.72 pentru scorul f1, ceea ce ne confirma ca dezghetarea straturilor 3 si 4 permit retelei sa se adapteze la specificul imaginilor. Observam imbunatatirile si in matricea de confuzie, darn v ramane dominant cu 1151 de exemple corecte. Totusi, avem mai putine fals pozitive decat mai devreme.

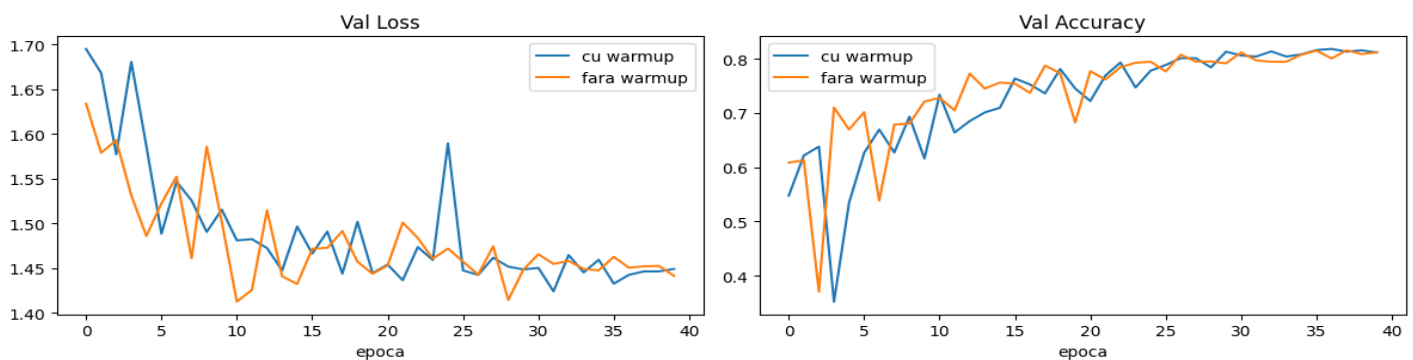
Matrice de confuzie - Alunite partial cu warmup



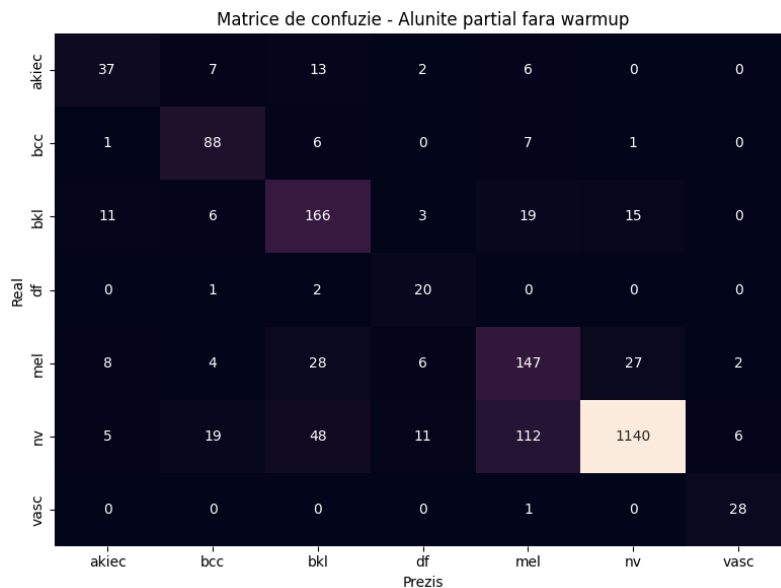
	precision	recall	f1-score	support
akiec	0.60	0.58	0.59	65
bcc	0.68	0.82	0.74	103
bkl	0.63	0.76	0.69	220
df	0.54	0.83	0.66	23
mel	0.54	0.68	0.60	222
nv	0.96	0.86	0.91	1341
vasc	0.74	0.97	0.84	29
accuracy			0.82	2003
macro avg	0.67	0.78	0.72	2003
weighted avg	0.84	0.82	0.83	2003

Din graficul de mai jos, partial unfreeze cu warmup si fara warmup converg similar. Fara warmup porneste mai brusc si converge mai rapid in primele 5 epoci, in timp ce varianta cu warmup prezinta o scadere brusca la epoca 3-4, fiind cauzata de calibrarea learning rete-ului. Fata de comparatia de mai sus, unde aveam rezultate similare, aici putem observa o diferenta mai mare, dar chiar si asa la final performanta ajunge sa fie cam la fel.

ResNet18 Alunite - partial unfreeze fara warmup - cu vs fara Warmup



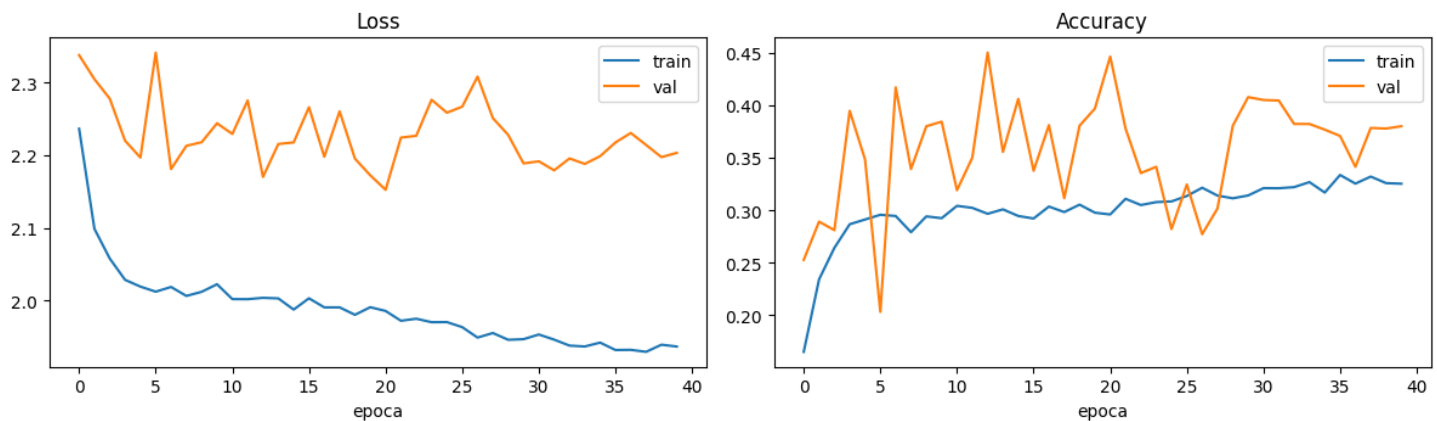
Performanta este asemanatoare, deci fie cu warmup, fie fara, obtinem cam aceleasi rezultate pe metrice si predictii.



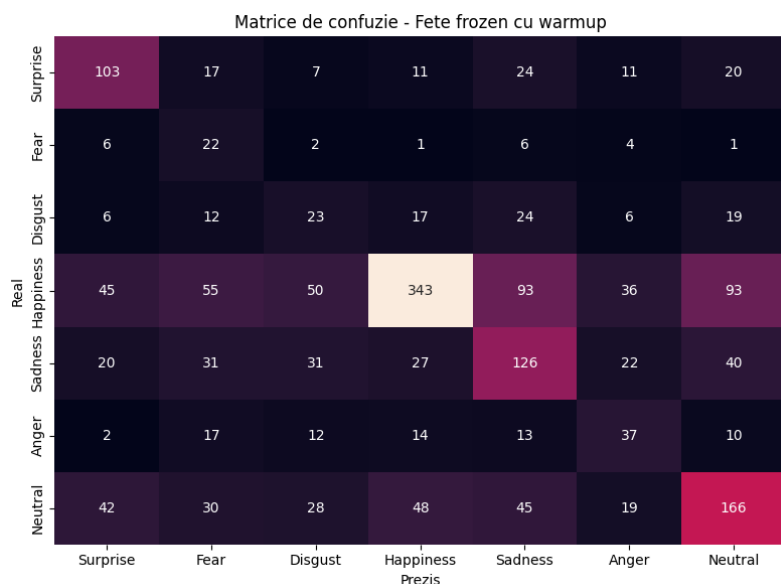
	precision	recall	f1-score	support
akiec	0.60	0.57	0.58	65
bcc	0.70	0.85	0.77	103
bkl	0.63	0.75	0.69	220
df	0.48	0.87	0.62	23
mel	0.50	0.66	0.57	222
nv	0.96	0.85	0.90	1341
vasc	0.78	0.97	0.86	29
accuracy			0.81	2003
macro avg	0.66	0.79	0.71	2003
weighted avg	0.84	0.81	0.82	2003

Observam un grafic mai ciudat, train converge constant, dar validare oscileaza si ramane sus pe parcursul celor 40 de epoci. Asta ne indica un overfit ridicat, backbone-ul inghetat nu reuseste sa extraga caracteristici suficiente de relevante pentru expresiile faciale, iar capul de clasificare memoreaza datele de antrenare fara sa generalizeze.

ResNet18 Fete - frozen cu warmup



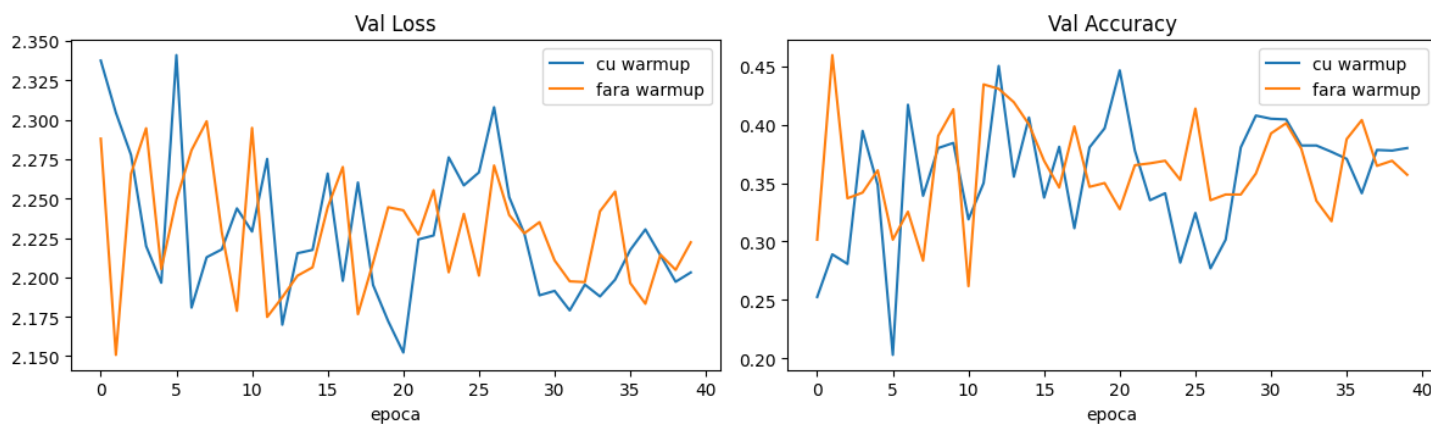
Avem o performanta foarte slaba pentru fete. Happiness este predominanta asa cum era de asteptat (343 exemple corecte), dar Fear si Disgust raman aproape nerecunoscute. Sunt generate false positive massive. Modelul tinde sa prezica Surprise, Fear si Happiness.



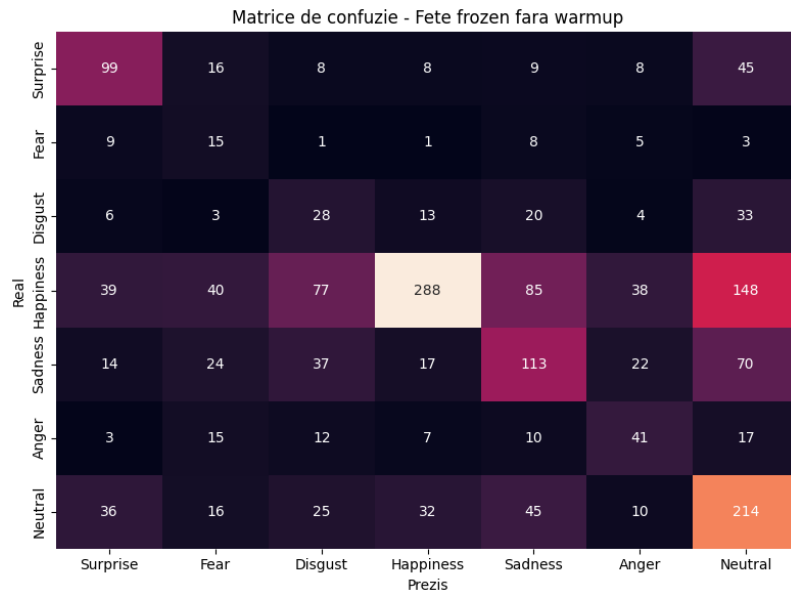
	precision	recall	f1-score	support
Surprise	0.46	0.53	0.49	193
Fear	0.12	0.52	0.19	42
Disgust	0.15	0.21	0.18	107
Happiness	0.74	0.48	0.58	715
Sadness	0.38	0.42	0.40	297
Anger	0.27	0.35	0.31	105
Neutral	0.48	0.44	0.46	378
accuracy			0.45	1837
macro avg	0.37	0.42	0.37	1837
weighted avg	0.52	0.45	0.47	1837

Ambele variante au comportament similar, la fel de slab. Dar aici la loss, cu warmup nu mai converge frumos, nu extrage suficiente caracteristici relevante.

ResNet18 Fete - frozen - cu vs fara Warmup



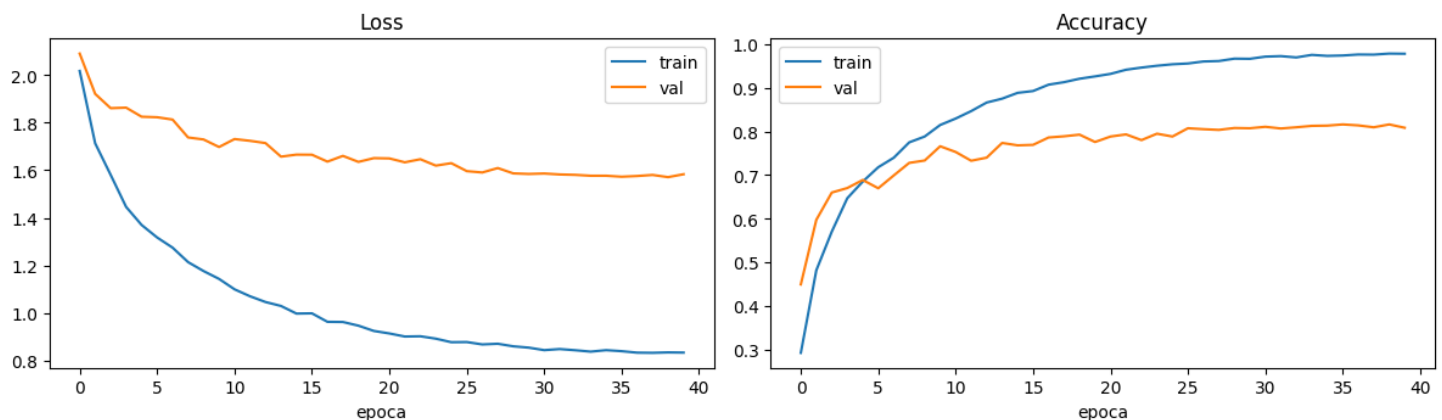
Observam ca erorile sunt sim ai mari, Happiness scade de la 343 la 288 exemple corecte. Confuda cu Neutral, 148 de exemple, dar obtine 214 de exemple corecte. Am obtinut o performanta scazuta fata de ce ma asteptam, expresiile faciale se dovedesc a avea nevoie de adaptarea straturilor lui ResNet-18.



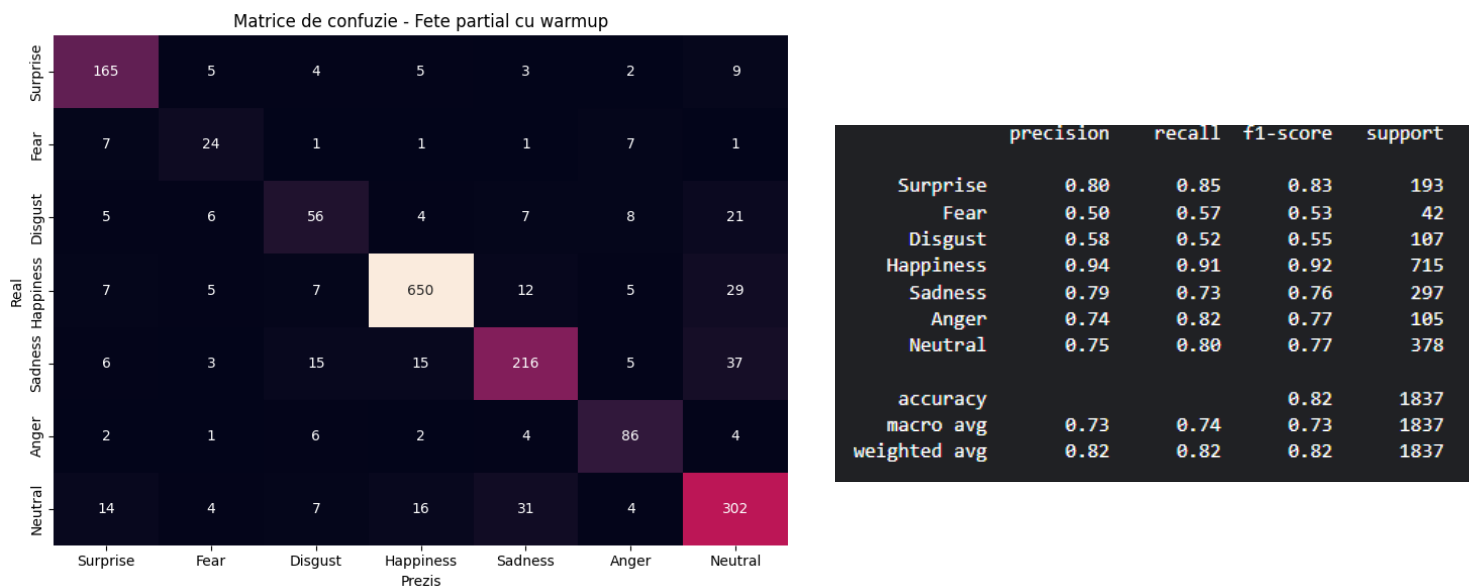
	precision	recall	f1-score	support
Surprise	0.48	0.51	0.50	193
Fear	0.12	0.36	0.18	42
Disgust	0.15	0.26	0.19	107
Happiness	0.79	0.40	0.53	715
Sadness	0.39	0.38	0.39	297
Anger	0.32	0.39	0.35	105
Neutral	0.40	0.57	0.47	378
accuracy			0.43	1837
macro avg	0.38	0.41	0.37	1837
weighted avg	0.53	0.43	0.45	1837

Pentru unfreeze pe 2 layer, am obtinut o configuratie mult mai buna, avem un overfit mai slab, dar inca e prezent asa cum putem vedea in grafic. In schimb, pentru acuratete modelul creste lin si e mult mai stabil si consistent. Nu avem oscilatii remarcabile, astfel aceasta varianta de model ne arata ca dezghetarea straturilor permite o antrenare mai lina sim ai buna.

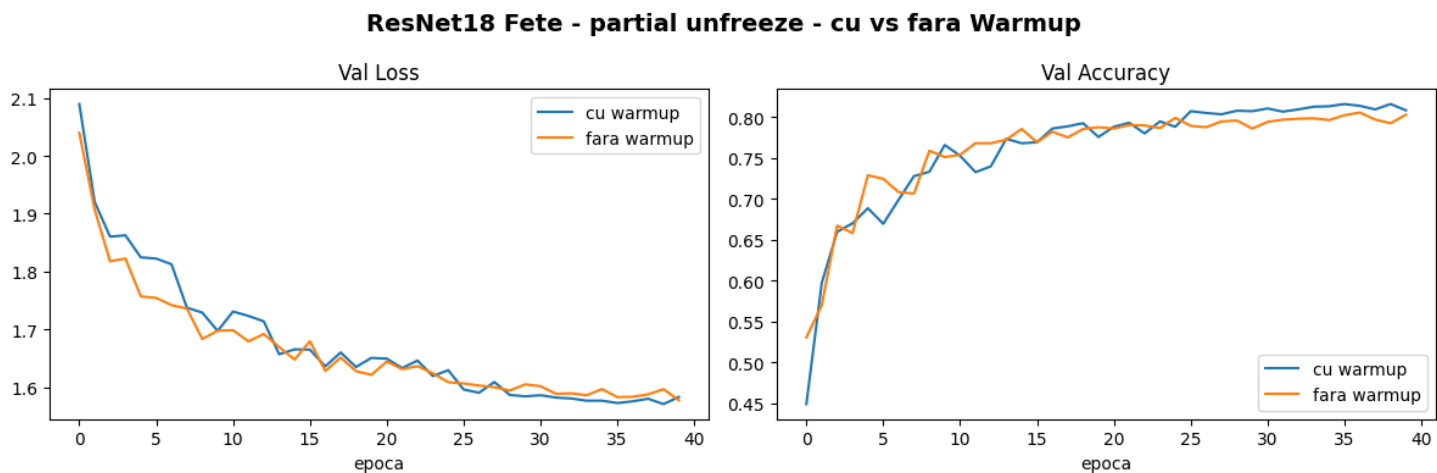
ResNet18 Fete - Partial unfreeze cu warmup



Rezultate mult mai bune, au evoluat fata de ce aveam mai sus. Happiness ajunge la 650 de exemple corecte, Neutral la 302. Chiar si clasele minoritare sunt imbunatatite. Confuziile raman intre Sadness si Neutral (37), intre Disgust si Neutral (21).



Variant afara warmup converge mai rapid, dar la final ambele modele ajung la rezultate similare, varianta cu warmup depaseste foarte putin modelul fara warmup. Astfel, unfreeze functioneaza bine pe ambele cazuri pentru fete.



Fara warmup, Happiness creste de la 650 la 652 exemple corecte, insa Neutral scade de la 302 la 280. Fear scade de la $f1 = 0.53$ la $f1 = 0.49$, iar Disgust de la 0.55 la 0.49, acestea fiind clasele minoritare, aratand ca sunt afectate mai mult fara warmup. In schimb, Sadness se imbunatateste (de la 216 ajunge la 225 corecte).

Matrice de confuzie - Fete partial fara warmup

Real \ Predict	Surprise	Fear	Disgust	Happiness	Sadness	Anger	Neutral
Surprise	161	4	4	3	6	1	14
Fear	7	23	1	1	6	3	1
Disgust	3	6	51	6	6	13	22
Happiness	5	6	12	652	11	3	26
Sadness	6	2	16	14	225	3	31
Anger	4	5	7	3	3	82	1
Neutral	12	6	10	22	42	6	280

	precision	recall	f1-score	support
Surprise	0.81	0.83	0.82	193
Fear	0.44	0.55	0.49	42
Disgust	0.50	0.48	0.49	107
Happiness	0.93	0.91	0.92	715
Sadness	0.75	0.76	0.76	297
Anger	0.74	0.78	0.76	105
Neutral	0.75	0.74	0.74	378
accuracy			0.80	1837
macro avg	0.70	0.72	0.71	1837
weighted avg	0.80	0.80	0.80	1837

Am observat ca freeze complet strica modelul, acesta obtine scoruri mult mai mici, decat modelul cu unfreeze pe layer 3 si 4. De asemenea, am vazut si diferenta intre cu/fara warmup care a adus un beneficiu pentru unfreeze. Pe freeze, warmup-ul este irelevant.

Totusi, am observat ca pentru fete am obtinut o performanta mai slaba decat la alunite, in ceea ce priveste modelul cu unfreeze pe 2 straturi. Acest lucru ar putea fi explicat prin faptul ce extragerea caracteristicilor de ResNet18 pe ImageNet este mai putin aplicabila pentru expresiile faciale.

Cel mai bun model ramane totusi unfreeze cu warmup pe ambele seturi de date.

Bonus

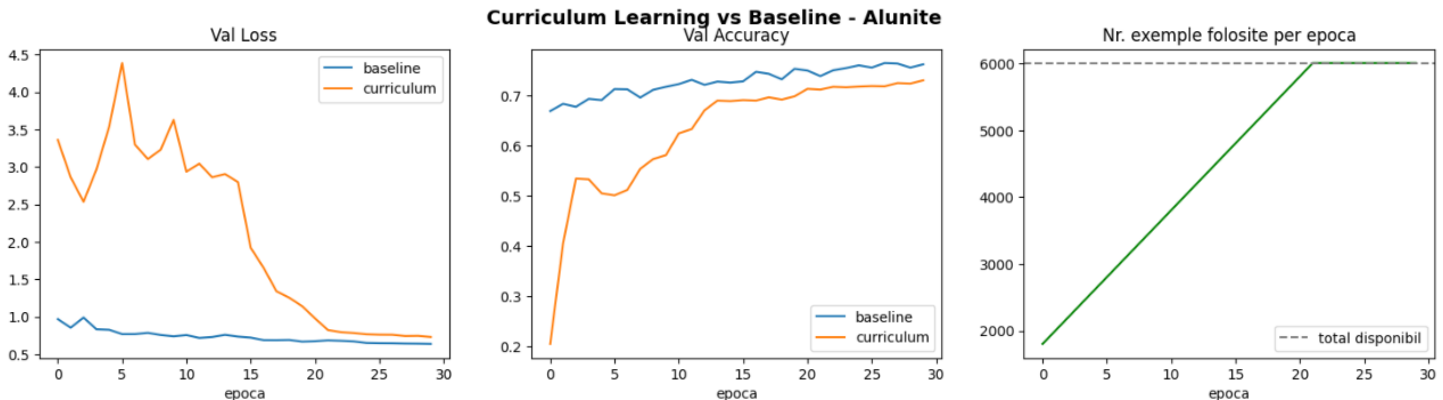
Practic modelul invata treptat, mai intai exemple usoare, iar complexitatea creste treptat in timp ce modelul se imbunatateste. Astfel, decidem cum defineste modelul ce e mai usor si ce e mai dificil. Am folosit self-paced learning, adica modelul insusi decide ce e mai greu pentru el. La inceputul fiecarei epoci, trecem toate imaginile prin modelul curent si calculam loss-ul individual pentru fiecare imagine. Un loss mare inseamna ca modelul greseste mult pe acea imagine, deci e mai dificila. Un loss mic inseamna ca modelul clasifica bine, deci e mai usoara. Dupa ce a calculat loss-urile, selectam doar imaginile cu loss sub un prag crescator. La primele epoci pragul e mai mic, dar pe parcursul antrenarii, pragul creste. Astfel, modelul nu e incarcat de exemple dificile, ci le primeste progresiv, cand e indeajuns de antrenat pentru a invata. Am folosit aceeasi arhitectura CNN, acelasi optimizer, acelasi scheduler.

Ca si inceput am antrenat un CNN standard pe intregul set de antrenare al alunitelor, cu exemple in ordine aleatoare la fiecare epoca (shuffle = true). Arhitectura, optimizer si scheduler sunt identice cu cele folosite in curriculum.

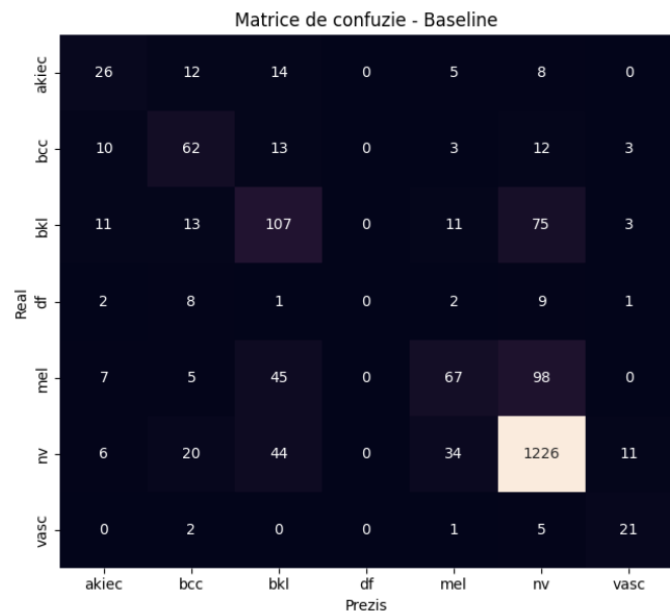
Pentru curriculum learning, la fiecare epoca, inainte de antrenare, se efectueaza urmatoorii pasi:

1. Calculul loss per imagine: toate imaginile sunt trecute prin augmentarile de validare si se calculeaza loss-ul per imagine cu `CrossEntropyLoss(reduction="none")`. Nu folosim augmentarile pentru ca ne dorim niste loss-uri stabile, nu influentate de variatii random.
2. Calcul prag: se calculeaza pragul dupa formula $f = \min(1.0, 0.3 + 0.7 * (nr_epoca / 21))$, adica la prima epoca foloseste 30% din date, iar dupa epoca 21 foloseste toate datele.
3. Selectare exemple: sunt retinute doar imaginile cu loss sub prag, adica cele pe care modelul le considera suficient de usoare la momentul respectiv. Apoi se formeaza un subset din care se construiesc DataLoader cu `shuffle = true`.
4. Antrenare: modelul e antrenat un singur pas pe subsetul selectat, apoi se reia pentru epoca urmatoare. Validarea se face pe setul complet de date. Antrenarea se face pe 30 de epoci.

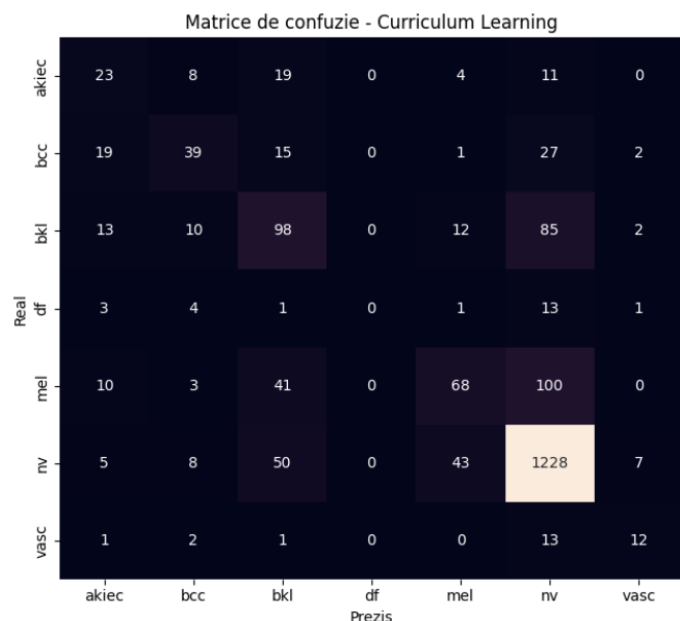
Mai jos, avem graficul pentru comparatie. Putem vedea inca din primele epoci diferente remarcabile. Varianta curriculum porneste cu un loss foarte mare care apoi scade brusc pe la epoca 15, cand a ajuns sa aiba destule exemple pentru antrenare. Spre final, cele 2 variante ajung sa converga similar, insa baseline-ul raman mai bun. La fel, observam si la acuratete, curriculum porneste cu o acuratete mult mai mica, dar recupereaza pana la final. Chiar si asa nu reuseste sa depaseasca baseline-ul.



In ceea ce priveste matricile de confuzie, observam ca ambele modele au o performanta similara. Ambele esueaza in a identifica clasa minoritara df, clasa care de-a lungul temei a fost cea mai greu de prezis. Totusi, baseline-ul are o performanta mai buna, nu cu mult, dar reuseste sa prezica mai bine pe celelalte clase.



	precision	recall	f1-score	support
akiec	0.48	0.35	0.41	65
bcc	0.62	0.44	0.51	103
bkl	0.47	0.50	0.48	220
df	0.00	0.00	0.00	23
mel	0.57	0.41	0.48	222
nv	0.85	0.92	0.89	1341
vasc	0.61	0.66	0.63	29
accuracy			0.76	2003
macro avg	0.51	0.47	0.49	2003
weighted avg	0.74	0.76	0.75	2003



	precision	recall	f1-score	support
akiec	0.33	0.14	0.20	65
bcc	0.46	0.41	0.43	103
bkl	0.39	0.39	0.39	220
df	0.00	0.00	0.00	23
mel	0.58	0.28	0.38	222
nv	0.81	0.94	0.87	1341
vasc	0.33	0.03	0.06	29
accuracy			0.73	2003
macro avg	0.42	0.31	0.33	2003
weighted avg	0.69	0.73	0.70	2003

Per total, acest experiment nu a adus o performanta mai buna fata de baseline pe setul de alunite (0.49 vs 0.33). Baseline-ul obtine scoruri mai bune si are o antrenare mai stabila. Acest lucru il putem explica prin faptul ca self-paced learning este mai sensibil la inceput, in primele epoci, modelul fiind slab, astfel se obtin rezultate nesigure, ceea ce poate duce la alegerea unui subset nereprezentat. Pe langa asta, datasetul este dezechilibrat, iar eliminarea exemplorlor cu loss mare in primele epoci inseamna de fapt eliminarea exacta a claselor rare si mai dificile, accentuand si mai mult dezechilibrul. Acest lucru este confirmat si de stagnarea acuratetei in primele 15 epoci.

Astfel, desi curriculum learning avea un mare potential in prima faza, implementarea sa necesita robustete pentru a obtine eficienta, de exemplu bazat mai bine pe caracteristicile imaginii, cum ar fi contrastul, decat sa ne bazam pe loss sau am fi putut combina cu tehnici de echilibrare a claselor pentru a preveni excluderea exemplelor rare in fazele de antrenare.

Scor Kaggle:

	Alunite	Fete
MLP	0.4120	0.55661
CNN	0.50126	0.64940
ResNet18	0.76913	0.74279
Bonus	0.4460	